

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO BÀI TẬP LỚN MÔN
HỆ THỐNG PHÂN TÁN**

**Đề tài: Phát triển hệ thống chat ngang hàng P2P
(Peer-to-Peer Chat System)**

Giảng viên:

Kim Ngọc Bách

Nhóm lớp:

03

Nhóm bài tập:

12

Vũ Thành Nam

B22DCCN568

Phạm Hồng Quang

B22DCCN652

Hà Nội - 2026

Lời cảm ơn

Nhóm chúng em xin gửi lời tri ân sâu sắc đến Quý Thầy/Cô tại Học viện Công nghệ Bưu chính Viễn thông nói chung và Khoa Công nghệ Thông tin nói riêng, vì đã tận tình giảng dạy và truyền đạt những nền tảng kiến thức quý báu trong suốt quá trình học tập. Đặc biệt, chúng em xin bày tỏ lòng biết ơn chân thành đến Thầy Kim Ngọc Bách – người đã trực tiếp hướng dẫn, đồng hành và hỗ trợ nhóm trong suốt quá trình thực hiện bài tập lớn này. Những lời chỉ bảo tận tâm, cùng phong thái làm việc nghiêm túc và tư duy nghiên cứu khoa học của Thầy không chỉ giúp nhóm hoàn thành đề tài mà còn là hành trang quý giá cho hành trình phát triển nghề nghiệp sau này. Kính chúc Thầy cùng Quý Thầy/Cô luôn dồi dào sức khỏe và gặt hái nhiều thành công trong sự nghiệp trồng người.

Mục lục

Lời cảm ơn.....	2
Chương I. Tổng quan đề tài.....	5
1.1. Đặt vấn đề.....	5
1.2 Mục tiêu của đề tài	5
1.3 Phạm vi nghiên cứu.....	5
1.4 Phương pháp nghiên cứu.....	5
Chương II. Cơ sở lý thuyết.....	5
2.1. Mô hình mạng ngang hàng (P2P).....	5
2.1.1. Định nghĩa.....	5
2.1.2 So sánh với mô hình Client-Server	5
2.1.3. Phân loại mạng P2P	6
2.2. Giao thức TCP (Transmission Control Protocol).....	6
2.3. Mã hóa AES-256-GCM	8
2.3.1. Tổng quan	8
2.3.2. Quy trình mã hóa trong hệ thống	9
2.4 Kiến trúc RESTful API	9
2.5. WebSocket và Socket.IO	10
2.5.1. WebSocket	10
2.5.2. Socket.IO	11
2.6. Cơ chế Store-and-Forward	12
Chương III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	14
3.1. Kiến trúc tổng quan hệ thống	14
3.1.1. Luồng giao tiếp chính	15
3.2 Thiết kế các thành phần.....	16
3.2.1. Bootstrap Server.....	16
3.2.2. Peer Node.....	20
3.3. Thiết kế cơ sở dữ liệu.....	23
3.3.1 Hệ quản trị cơ sở dữ liệu.....	23
3.3.2. Chiến lược lưu trữ kép (Dual Storage Strategy)	23
3.3.3. Sơ đồ quan hệ thực thể (ER Diagram).....	24
3.3.4. Mô tả chi tiết từng bảng.....	24
3.3.5. Tổng hợp quan hệ giữa các bảng	26
3.4. Thiết kế Giao thức và các Luồng xử lý chính.....	26

3.4.1. Giao thức truyền tin (Wire Format)	26
3.4.2. Luồng gửi tin nhắn trực tiếp (Direct Message).....	27
3.4.3. Cơ chế ACK và Retry	27
3.4.4. Luồng xử lý tin nhắn Offline (Store-and-Forward)	28
3.4.5. Luồng Relay tin nhắn (Trung gian)	30
3.4.6. Luồng truyền tải File (TCP File Transfer).....	32
3.4.7. Luồng Discovery và Heartbeat	33
CHƯƠNG IV : CÀI ĐẶT VÀ TRIỂN KHAI.....	35
4.1. Công nghệ sử dụng.....	35
4.2. Cấu trúc mã nguồn	36
4.3. Cấu hình môi trường và cơ sở dữ liệu.....	37
4.3.1. Chi tiết các biến môi trường cấu hình.....	37
4.3.2. Cấu trúc Cơ sở dữ liệu (MySQL Database Schema)	38
4.4. Quy trình cài đặt chi tiết.....	39
4.5. Khởi chạy và vận hành hệ thống	40
4.5.1. Cách 1: Sử dụng Bootstrap Launcher UI.....	40
4.5.2. Cách 2: Khởi chạy thủ công thông qua Terminal	40
CHƯƠNG V : KẾT QUẢ VÀ DEMO.....	41
5.1 Giao diện Bootstrap Launcher	41
5.2 Giao diện Peer Web UI	41
5.3. Demo gửi tin nhắn trực tiếp (Direct Message).....	42
5.4. Demo gửi tin nhắn nhóm (Group Message).....	43
5.5 Demo Broadcast Message	43
5.6 Demo gửi file	44
5.7. Demo Offline Queue (Store-and-Forward).....	45
5.8. Tổng hợp kết quả kiểm thử	45
CHƯƠNG 6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	47
6.1. Kết Quả Đạt Được.....	47
6.2. Hạn Chế.....	47
6.3. Hướng Phát Triển	48
Tài liệu tham khảo	49

Chương I. Tổng quan đề tài

Chương II. Cơ sở lý thuyết

2.1. Mô hình mạng ngang hàng (P2P)

2.1.1. Định nghĩa

Mạng ngang hàng (Peer-to-Peer – P2P) là một mô hình mạng phân tán, trong đó các thiết bị tham gia mạng (gọi là peer) hoạt động với vai trò ngang nhau và không phụ thuộc hoàn toàn vào máy chủ trung tâm. Trong mô hình này, mỗi peer vừa có khả năng cung cấp tài nguyên như dữ liệu, băng thông hoặc dịch vụ cho các peer khác, vừa có thể sử dụng tài nguyên được chia sẻ từ các peer trong mạng.

Kiến trúc P2P giúp tận dụng tối đa tài nguyên của các thiết bị tham gia, tăng khả năng mở rộng hệ thống và giảm tải cho máy chủ trung tâm. Mô hình này thường được ứng dụng trong các hệ thống chia sẻ tệp tin, truyền dữ liệu phân tán, blockchain và các ứng dụng truyền thông trực tuyến.

2.1.2 So sánh với mô hình Client-Server

Để làm rõ bản chất phân tán của mạng P2P, Bảng 2.1 đối chiếu mô hình này với mô hình Client-Server truyền thống trên các tiêu chí kiến trúc, vai trò node, khả năng chịu lỗi và khả năng mở rộng.

Bảng 2.1: So sánh mô hình Client-Server và Peer-to-Peer

Tiêu chí	Client-Server	Peer-to-Peer
Kiến trúc	Tập trung, phụ thuộc vào server trung tâm	Phân tán, không có node trung tâm điều phối
Vai trò node	Client chỉ tiêu thụ dịch vụ; Server chỉ cung cấp	Mỗi node đồng thời vừa cung cấp vừa tiêu thụ tài nguyên
Điểm lỗi	Server là <i>single point of failure</i>	Không tồn tại <i>single point of failure</i>
Khả năng mở rộng	Bị giới hạn bởi năng lực server	Tự mở rộng khi số lượng peer tăng
Lưu trữ dữ liệu	Tập trung trên server	Phân tán trên nhiều peer
Ví dụ điển hình	Web server, Email server	BitTorrent, Blockchain, WebRTC

Sự khác biệt cốt lõi nằm ở khả năng chịu lỗi và tính mở rộng: trong khi mô hình Client-Server phụ thuộc hoàn toàn vào sự sẵn sàng của server, mô hình P2P phân phối tải và trách nhiệm đồng đều giữa các peer, giúp hệ thống hoạt động ổn định ngay cả khi một số node ngừng hoạt động.

2.1.3. Phân loại mạng P2P

Dựa trên mức độ tập trung hóa và cơ chế tổ chức mạng, kiến trúc P2P được phân thành ba loại chính:

1. Pure P2P (P2P thuần túy): Không có bất kỳ server trung tâm nào; toàn bộ peer hoàn toàn bình đẳng về chức năng và quyền hạn. Mô hình này đạt mức phân tán cao nhất nhưng gặp khó khăn trong việc tìm kiếm tài nguyên hiệu quả. Ví dụ điển hình: Gnutella.
2. Hybrid P2P (P2P lai): Kết hợp một server trung tâm đảm nhiệm chức năng hỗ trợ tìm kiếm và thiết lập kết nối ban đầu, trong khi dữ liệu thực tế vẫn được trao đổi trực tiếp giữa các peer. Đây là sự dung hòa giữa hiệu quả vận hành và tính phân tán. Ví dụ điển hình: Napster, BitTorrent với tracker.
3. Structured P2P (P2P có cấu trúc): Áp dụng cơ chế DHT (Distributed Hash Table) để tổ chức và định vị tài nguyên một cách có hệ thống, cho phép truy cập hiệu quả mà không cần server trung tâm. Ví dụ điển hình: Chord, Kademlia.

Hệ thống được xây dựng trong đồ án này thuộc loại Hybrid P2P: bootstrap server đóng vai trò tracker, chịu trách nhiệm hỗ trợ peer discovery và lưu trữ metadata; tuy nhiên, toàn bộ luồng tin nhắn chat được truyền trực tiếp giữa các peer thông qua kết nối TCP, đảm bảo tính phân tán trong trao đổi dữ liệu thực tế.

2.2. Giao thức TCP (Transmission Control Protocol)

TCP là giao thức truyền thông hướng kết nối (connection-oriented) hoạt động ở tầng Transport trong mô hình TCP/IP. Khác với UDP, TCP thiết lập một kênh truyền tin đáng tin cậy giữa hai đầu cuối trước khi bắt đầu trao đổi dữ liệu thông qua cơ chế bắt tay ba bước (three-way handshake). TCP đảm bảo ba đặc tính cốt lõi sau:

- Độ tin cậy (Reliability): Mọi gói tin đều được xác nhận (ACK) bởi phía nhận. Nếu không nhận được xác nhận trong khoảng thời gian quy định, gói tin sẽ được truyền lại tự động, đảm bảo dữ liệu đến đích đúng thứ tự và không bị mất mát.
- Kiểm soát luồng (Flow Control): TCP sử dụng cơ chế cửa sổ trượt (sliding window) để điều chỉnh tốc độ gửi phù hợp với năng lực xử lý của bên nhận, tránh tình trạng tràn bộ đệm.
- Kiểm soát tắc nghẽn (Congestion Control): Khi phát hiện tắc nghẽn trên đường truyền (thông qua mất gói hoặc tăng RTT), TCP tự động giảm tốc độ gửi theo các thuật toán như Slow Start, AIMD (Additive Increase Multiplicative Decrease), giúp ổn định toàn bộ mạng.

Ứng dụng TCP trong hệ thống P2P Chat:

TCP được chọn làm giao thức truyền tải nền tảng cho hệ thống P2P Chat vì tính đáng tin cậy là yêu cầu bắt buộc đối với ứng dụng nhắn tin — mỗi tin nhắn phải được đảm bảo đến tay người nhận, đúng thứ tự, và không bị thất lạc hay trùng lặp.

Trong kiến trúc hệ thống, mỗi peer đồng thời đóng hai vai trò:

- TCP Server: Lắng nghe kết nối đến (inbound) từ các peer khác trên một cổng cố định, sẵn sàng nhận tin nhắn bất kỳ lúc nào.
- TCP Client: Chủ động khởi tạo kết nối (outbound) đến peer đích khi có nhu cầu gửi tin nhắn.

Định dạng khung tin (Frame Format)

Để đảm bảo khả năng phân tách và phân tích cú pháp (*parsing*) tin nhắn một cách chính xác trên luồng TCP liên tục, hệ thống áp dụng định dạng **JSON Newline-Delimited** — mỗi tin nhắn là một đối tượng JSON hoàn chỉnh kết thúc bằng ký tự xuống dòng `\n`. Cách tiếp cận này cho phép bên nhận đọc từng dòng và giải mã độc lập mà không cần biết trước kích thước gói tin.

Cấu trúc của một khung tin nhắn trực tiếp (*direct message*) được định nghĩa như sau:

```
json
{
  "type": "direct_message",
  "messageId": "uuid-v4",
  "fromPeerId": "peer-a",
  "fromUsername": "Alice",
  "toPeerId": "peer-b",
  "encrypted": true,
  "encryption": {
    "algorithm": "aes-256-gcm",
    "iv": "base64...",
    "tag": "base64...",
    "data": "base64..."
  },
  "createdAt": "2026-05-20T10:00:00.000Z"
}
```

Listing 2.1: Cấu trúc frame TCP – tin nhắn trực tiếp

Ý nghĩa các trường trong khung tin được mô tả trong Bảng 2.2 dưới đây:

Bảng 2.2: Mô tả các trường trong frame tin nhắn TCP

Trường	Kiểu dữ liệu	Mô tả
type	String	Loại tin nhắn, xác định cách xử lý phía nhận (direct_message, ack, v.v.)
messageId	String (UUID v4)	Định danh duy nhất của tin nhắn, phục vụ xác nhận và chống trùng lặp
fromPeerId	String	Định danh của peer gửi
fromUsername	String	Tên hiển thị của người gửi
toPeerId	String	Định danh của peer nhận
encrypted	Boolean	Cờ đánh dấu nội dung có được mã hóa hay không
encryption.algorithm	String	Thuật toán mã hóa được sử dụng (aes-256-gcm)
encryption.iv	String (Base64)	Vector khởi tạo (Initialization Vector) cho AES-GCM
encryption.tag	String (Base64)	Thẻ xác thực (Authentication Tag) đảm bảo tính toàn vẹn
encryption.data	String (Base64)	Nội dung tin nhắn đã được mã hóa
createdAt	String (ISO 8601)	Thời điểm tạo tin nhắn theo chuẩn UTC

Việc kết hợp AES-256-GCM trong tầng ứng dụng với độ tin cậy của TCP ở tầng Transport giúp hệ thống đảm bảo đồng thời cả **tính bảo mật** (*confidentiality*), **tính toàn vẹn** (*integrity*) và **tính sẵn sàng** (*reliability*) của mỗi tin nhắn được trao đổi giữa các peer.

2.3. Mã hóa AES-256-GCM

2.3.1. Tổng quan

AES (Advanced Encryption Standard) là thuật toán mã hóa đối xứng tiêu chuẩn, được NIST chấp nhận từ năm 2001. AES-256 sử dụng khóa 256-bit, cung cấp mức độ bảo mật rất cao. GCM (Galois/Counter Mode) là chế độ vận hành kết hợp cả mã hóa (confidentiality) và xác thực (authentication) trong cùng một phép toán, gọi là *Authenticated Encryption with Associated Data* (AEAD).

2.3.2. Quy trình mã hóa trong hệ thống

1. **Derive Key:** Từ shared secret, dùng SHA-256 để tạo khóa 256-bit cố định.
2. **Encrypt:** Tạo IV ngẫu nhiên 12 bytes, mã hóa plaintext bằng AES-256-GCM.
3. **Output:** Trả về iv, tag (authentication tag), và data (ciphertext), tất cả đều ở dạng Base64.
4. **Decrypt:** Bên nhận dùng cùng shared secret để derive key, giải mã bằng IV và xác thực bằng tag.

2.4 Kiến trúc RESTful API

REST (Representational State Transfer) là kiểu kiến trúc phần mềm được Roy Fielding đề xuất năm 2000, định nghĩa một tập hợp các ràng buộc thiết kế cho hệ thống phân tán dựa trên HTTP. Các ràng buộc cốt lõi của REST bao gồm tính phi trạng thái (stateless) — mỗi request chứa đầy đủ thông tin để server xử lý mà không cần nhớ trạng thái phiên trước — và giao diện đồng nhất (uniform interface) thông qua các HTTP method chuẩn (GET, POST, PUT, DELETE).

Trong hệ thống P2P Chat, **REST API** được triển khai ở hai tầng riêng biệt với vai trò khác nhau, sử dụng Express.js — framework web phổ biến nhất cho Node.js nhờ kiến trúc middleware linh hoạt và hiệu năng cao.

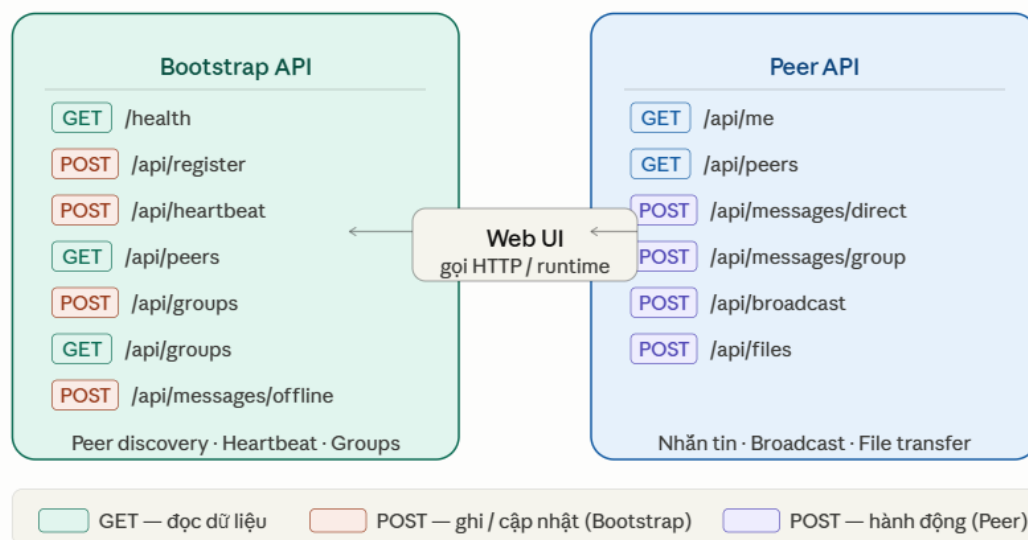
Bootstrap Server API đóng vai trò là điểm tập trung duy nhất trong hệ thống Hybrid P2P, cung cấp các endpoint phục vụ vòng đời của một peer trong mạng:

- POST /api/register — Peer đăng ký vào mạng, cung cấp thông tin định danh và địa chỉ lắng nghe TCP.
- POST /api/heartbeat — Peer gửi tín hiệu định kỳ để duy trì trạng thái online; Bootstrap Server dùng cơ chế này để phát hiện peer ngắt kết nối.
- GET /api/peers — Truy vấn danh sách peer đang hoạt động, phục vụ peer discovery.
- POST /api/groups / GET /api/groups — Tạo và truy vấn danh sách nhóm chat.
- POST /api/messages/offline — Lưu tin nhắn cho peer hiện đang offline, phục vụ cơ chế offline queue.
- GET /health — Kiểm tra trạng thái server, thường dùng cho load balancer hoặc monitoring.

Peer API là REST API nội bộ chạy trên mỗi peer, phục vụ hai mục đích chính: cho phép Web UI tương tác với peer thông qua HTTP thay vì giao tiếp trực tiếp với tầng TCP, và cung cấp giao diện lập trình cho các tích hợp runtime:

- GET /api/me — Trả về thông tin định danh của peer hiện tại.
- GET /api/peers — Danh sách peer đã biết trong mạng cục bộ.
- POST /api/messages/direct — Gửi tin nhắn trực tiếp đến một peer cụ thể qua kênh TCP.
- POST /api/messages/group — Gửi tin nhắn đến toàn bộ thành viên trong một nhóm.
- POST /api/broadcast — Phát tin đến tất cả peer đang kết nối.
- POST /api/files — Khởi tạo phiên truyền file đến peer đích.

Sơ đồ Hình 2.2 minh họa bản đồ API của hai thành phần, cùng với luồng gọi từ Web UI và runtime qua giao thức HTTP.



Hình 2.2: Bản đồ API của Bootstrap Server và Peer App

2.5. WebSocket và Socket.IO

2.5.1. WebSocket

WebSocket là giao thức truyền thông hai chiều (full-duplex) được chuẩn hóa trong RFC 6455, hoạt động trên một kết nối TCP duy nhất. Điểm khác biệt căn bản so với HTTP truyền thống nằm ở mô hình giao tiếp: HTTP hoạt động theo cơ chế request-response — client luôn phải chủ động gửi request và server chỉ phản hồi khi được hỏi. WebSocket xóa bỏ ràng buộc này bằng cách thiết lập một kênh truyền tin liên tục (persistent connection), cho phép cả hai phía chủ động gửi dữ liệu bất kỳ lúc nào mà không cần thêm overhead của HTTP header.

Quá trình thiết lập kết nối WebSocket bắt đầu bằng một HTTP handshake thông thường (*Upgrade request*), sau đó giao thức được nâng cấp (*protocol upgrade*) sang WebSocket. Từ thời điểm đó, kết nối TCP bên dưới được tái sử dụng hoàn toàn cho việc truyền tải WebSocket frame — loại bỏ chi phí thiết lập lại kết nối cho mỗi lần trao

đổi dữ liệu. So sánh hai mô hình được trình bày trong Bảng 2.4.

Bảng 2.4: So sánh HTTP polling và WebSocket

Tiêu chí	HTTP polling	WebSocket
Hướng truyền	Client → Server (một chiều mỗi request)	Hai chiều đồng thời
Kết nối	Thiết lập lại mỗi request	Một kết nối duy nhất, duy trì liên tục
Độ trễ	Cao (phụ thuộc tần suất polling)	Thấp (gần như tức thời)
Overhead	HTTP header cho mỗi request (~800 byte)	WebSocket frame header (2–14 byte)
Phù hợp	Dữ liệu cập nhật không thường xuyên	Realtime, tần suất cao

2.5.2. Socket.IO

Socket.IO là thư viện JavaScript xây dựng trên nền WebSocket, bổ sung một lớp trừu tượng với nhiều tính năng quan trọng mà WebSocket thuần không có:

- Tự động fallback: Khi môi trường mạng không hỗ trợ WebSocket (tường lửa chặn, proxy không tương thích), Socket.IO tự động chuyển sang HTTP long-polling mà không cần thay đổi code ứng dụng, đảm bảo tính khả dụng cao.
- Room và Namespace: Cho phép phân nhóm các kết nối theo logic nghiệp vụ. Namespace tạo ra các kênh truyền độc lập trên cùng một server; Room cho phép gửi sự kiện đến một tập con kết nối trong namespace đó. Đây là cơ chế nền tảng để triển khai tính năng nhóm chat.
- Tự động reconnect: Socket.IO tích hợp sẵn cơ chế phát hiện ngắt kết nối và tự động thử kết nối lại theo chiến lược exponential backoff, tránh làm quá tải server khi nhiều client cùng reconnect đồng thời.
- Event-based API: Thay vì làm việc trực tiếp với dữ liệu nhị phân như WebSocket thuần, Socket.IO cung cấp mô hình lập trình dựa trên sự kiện có tên (named events) — tương tự EventEmitter trong Node.js — giúp code rõ ràng và dễ bảo trì hơn đáng kể.

Ứng dụng trong hệ thống P2P Chat

Socket.IO được sử dụng cho kênh truyền thông giữa peer server (Node.js)

và Web UI (trình duyệt), phục vụ bốn luồng dữ liệu realtime chính:

- **Tin nhắn đến:** Khi peer nhận được tin nhắn qua kênh TCP từ peer khác, sự kiện được đẩy ngay lập tức lên Web UI để hiển thị mà không cần client polling.
- **Trạng thái peer:** Các thay đổi trạng thái *online/offline* của peer trong mạng được phát broadcast đến Web UI ngay khi Bootstrap Server phát hiện.
- **Log hệ thống:** Luồng log runtime từ peer server được stream liên tục lên giao diện developer console trong Web UI, hỗ trợ debug và giám sát.
- **Thông kê và metrics:** Các chỉ số như số peer đang kết nối, lưu lượng tin nhắn, và trạng thái kết nối TCP được cập nhật định kỳ lên dashboard Web UI.

2.6. Cơ chế Store-and-Forward

Store-and-forward (lưu rồi chuyển tiếp) là kỹ thuật kinh điển trong các hệ thống truyền thông phân tán, trong đó tin nhắn được lưu tạm trên một node trung gian khi node đích chưa sẵn sàng nhận. Kỹ thuật này có nguồn gốc từ các mạng chuyển mạch gói (packet-switched networks) và ngày nay được ứng dụng rộng rãi trong email (SMTP), hệ thống nhắn tin di động (SMS), và các nền tảng chat phi tập trung.

Cơ chế này giải quyết một thách thức cốt lõi của mạng P2P: không giống môi trường tập trung nơi server luôn sẵn sàng tiếp nhận, trong mạng P2P các peer có thể ngắt kết nối bất kỳ lúc nào. Nếu không có store-and-forward, tin nhắn gửi đến peer offline sẽ bị mất vĩnh viễn.

Quy trình xử lý trong hệ thống P2P Chat

Hệ thống triển khai store-and-forward theo ba cấp độ ưu tiên giảm dần, đảm bảo tin nhắn luôn được phân phối ngay cả trong điều kiện mạng không ổn định.

Cấp 1 — Gửi trực tiếp qua TCP (Direct Delivery): Peer A thực hiện kết nối TCP trực tiếp đến địa chỉ của Peer C lấy từ Bootstrap Server. Nếu kết nối thành công, tin nhắn được truyền ngay lập tức và quy trình kết thúc. Nếu kết nối thất bại (timeout, connection refused), hệ thống thực hiện retry theo chiến lược exponential backoff — tránh gây quá tải mạng bằng cách tăng dần thời gian chờ giữa các lần thử. Sau khi vượt quá ngưỡng retry cho phép mà vẫn thất bại, hệ thống chuyển sang cấp 2.

Cấp 2 — Chuyển tiếp qua peer trung gian (Relay): Peer A yêu cầu Bootstrap Server cung cấp danh sách các peer đang online, sau đó chọn một peer làm node trung gian để relay tin nhắn đến Peer C. Cơ chế này hoạt động hiệu quả khi Peer C thực sự đang online nhưng không thể nhận kết nối trực tiếp từ Peer A do hạn chế NAT hoặc tường lửa. Nếu relay cũng thất bại — do không có peer trung gian phù hợp hoặc Peer C đã thực sự offline — hệ thống chuyển sang cấp 3.

Cấp 3 — Lưu trữ trên Bootstrap Server (Offline Queue): Peer A gửi tin nhắn đến Bootstrap Server qua REST API (POST /api/messages/offline). Bootstrap Server lưu tin nhắn vào hàng đợi offline của Peer C kèm timestamp và trạng thái

pending. Tin nhắn được giữ trong queue cho đến khi Peer C quay lại online.

Phân phối lại khi Peer C online: Sau khi Peer C khởi động và đăng ký lại với Bootstrap Server, peer thực hiện poll offline queue (GET /api/messages/offline). Mỗi tin nhắn được tải về, giải mã, và hiển thị theo đúng thứ tự timestamp. Sau khi nhận thành công, Peer C gửi xác nhận để Bootstrap Server đánh dấu trạng thái delivered và xóa khỏi queue, tránh giao lại trùng lặp.

Bảng 2.5: Tóm tắt ba cấp độ phân phối tin nhắn

Cấp độ	Phương thức	Điều kiện kích hoạt	Kết quả khi thất bại
1	TCP trực tiếp	Mặc định cho mọi tin nhắn	Chuyển sang cấp 2
2	Relay qua peer trung gian	TCP trực tiếp thất bại sau retry	Chuyển sang cấp 3
3	Offline queue trên Bootstrap	Relay thất bại hoặc không có peer trung gian	Lưu đến khi peer online

Chương III. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng quan hệ thống

Hệ thống P2P Chat được thiết kế theo mô hình ngang hàng có hỗ trợ (Assisted Peer-to-Peer), trong đó các peer giao tiếp trực tiếp với nhau qua kết nối TCP, trong khi một máy chủ trung tâm đóng vai trò hỗ trợ phụ trợ nhưng không tham gia vào luồng tin nhắn chính. Kiến trúc này đảm bảo hiệu năng cao, độ trễ thấp và giảm tải cho máy chủ trung tâm.

Hệ thống bao gồm hai thành phần chính:

Thành phần 1 — Bootstrap Server (Máy chủ khởi động)

Bootstrap Server hoạt động như một tracker trung tâm, cung cấp các dịch vụ hỗ trợ không đồng bộ cho toàn bộ mạng lưới peer. Cụ thể, máy chủ này thực hiện các chức năng sau:

- **Peer Discovery (Khám phá peer):** Cung cấp REST API cho phép các peer đăng ký địa chỉ IP, cổng TCP và trạng thái hoạt động. Khi một peer muốn kết nối với peer khác, nó truy vấn Bootstrap Server để lấy thông tin địa chỉ tương ứng.
- **Quản lý trạng thái (State Management):** Theo dõi trạng thái online/offline của từng peer theo thời gian thực. Thông tin này được cập nhật định kỳ thông qua cơ chế heartbeat hoặc khi peer chủ động thông báo thay đổi trạng thái.
- **Group Metadata:** Lưu trữ và cung cấp thông tin về các nhóm chat, bao gồm danh sách thành viên, tên nhóm và các siêu dữ liệu liên quan.
- **Offline Message Queue (Hàng đợi tin nhắn ngoại tuyến):** Khi peer đích đang offline, tin nhắn được lưu tạm thời vào hàng đợi trên Bootstrap Server. Khi peer đích kết nối trở lại, tin nhắn sẽ được chuyển tiếp đến đích.
- **Launcher UI:** Cung cấp giao diện web khởi động để quản trị viên hoặc người dùng có thể khởi chạy và giám sát các peer trong hệ thống.

Bootstrap Server được triển khai dưới dạng dịch vụ RESTful, tiếp nhận các yêu cầu HTTP từ các peer và phản hồi dữ liệu dưới định dạng JSON.

Thành phần 2 — Peer App (Ứng dụng Peer)

Mỗi peer trong hệ thống là một ứng dụng Node.js độc lập, hoàn toàn tự trị và có khả năng hoạt động song song với nhiều peer khác. Mỗi Peer App bao gồm ba module chức năng cốt lõi:

- **Web UI:** Giao diện người dùng dựa trên trình duyệt web, cho phép người dùng soạn và xem tin nhắn, quản lý danh sách liên lạc và theo dõi trạng thái kết nối.
- **TCP Server:** Mỗi peer mở một cổng TCP lắng nghe (listening socket) để nhận kết nối đến từ các peer khác. Khi có tin nhắn gửi đến, TCP Server xử lý và đẩy dữ liệu lên lớp ứng dụng để hiển thị lên Web UI.
- **TCP Client:** Module này chịu trách nhiệm thiết lập kết nối ra ngoài đến

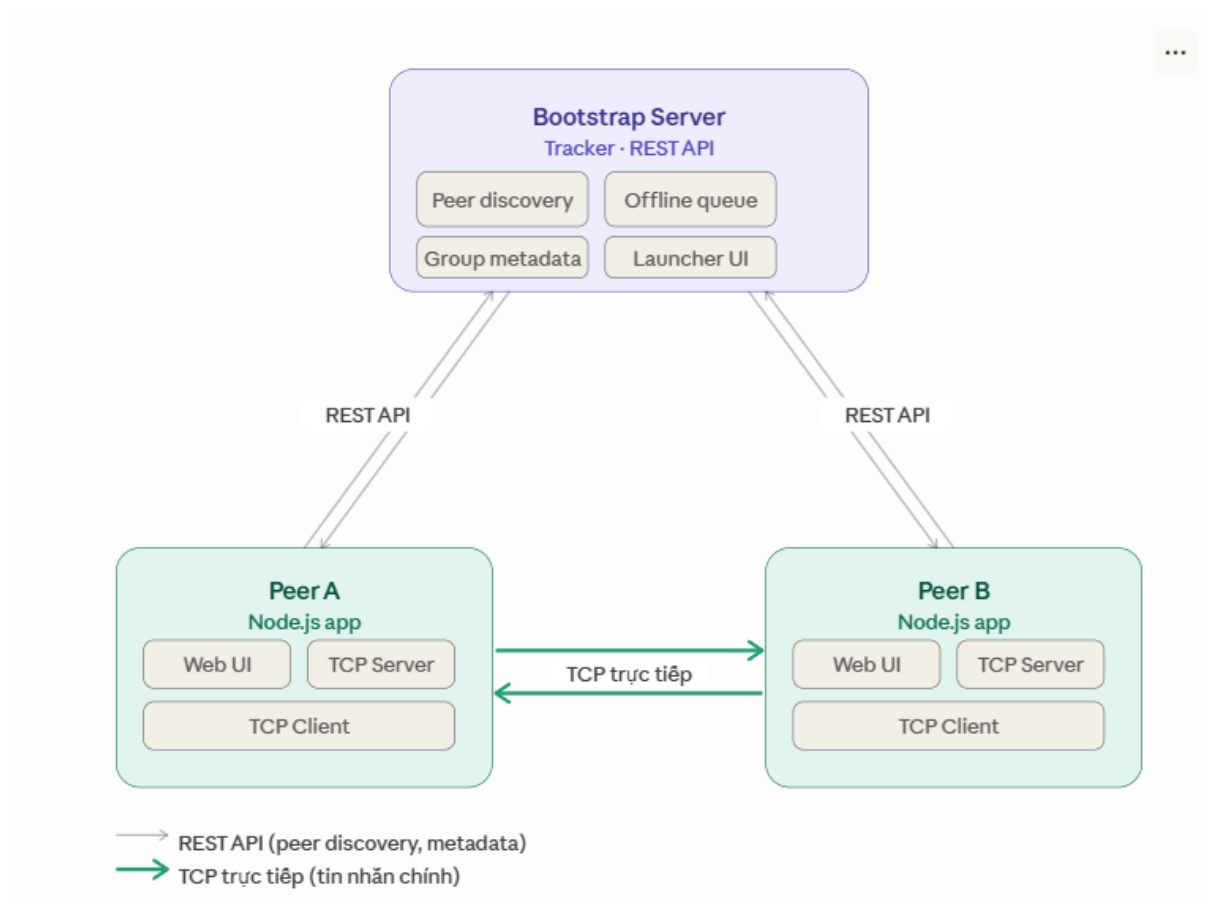
TCP Server của peer đích khi người dùng muốn gửi tin nhắn. Địa chỉ kết nối được tra cứu từ Bootstrap Server trước khi thiết lập phiên TCP.

Điểm quan trọng về luồng truyền dữ liệu

Điểm đặc trưng và quan trọng nhất của kiến trúc này là tin nhắn chat chính được truyền trực tiếp giữa các peer qua kết nối TCP, hoàn toàn không đi qua Bootstrap Server. Điều này mang lại các ưu điểm sau:

- Độ trễ truyền tải thấp hơn so với mô hình client-server truyền thống, do tin nhắn không phải trung chuyển qua máy chủ trung gian.
- Bootstrap Server không trở thành điểm nghẽn cổ chai (bottleneck) khi số lượng peer và lưu lượng tin nhắn tăng cao.
- Khả năng mở rộng (scalability) tốt hơn: thêm peer mới không làm tăng tải đáng kể lên Bootstrap Server.

Bootstrap Server chỉ tham gia vào các luồng phụ trợ như đăng ký peer, tra cứu địa chỉ, đồng bộ metadata nhóm và lưu trữ tin nhắn cho peer ngoại tuyến.



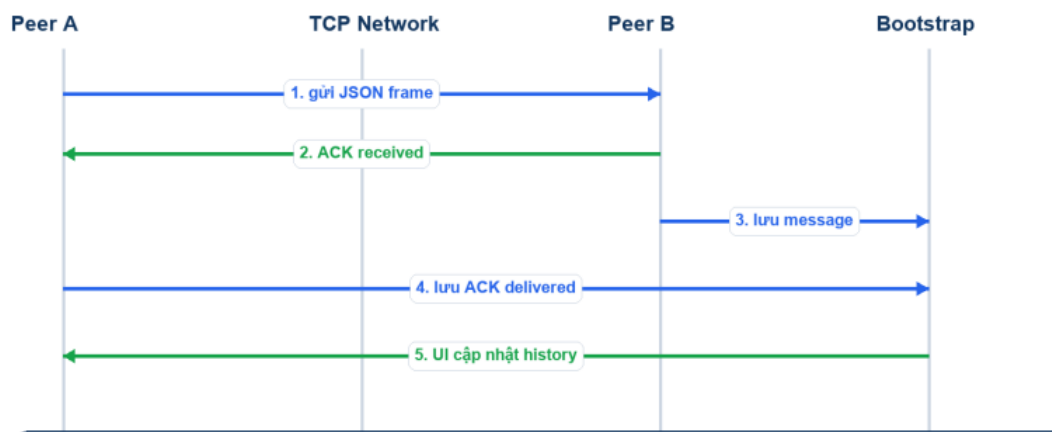
Hình 3.1: Kiến trúc tổng quan hệ thống P2P Chat

3.1.1. Luồng giao tiếp chính

Khi peer A gửi tin nhắn cho Peer B :

1. Peer A tra cứu thông tin Peer B (host, TCP port) từ danh sách peer đã sync.
2. Peer A mở kết nối TCP đến Peer B.
3. Peer A gửi payload JSON (có thể đã mã hóa AES-256-GCM).
4. Peer B nhận, giải mã, lưu tin nhắn và trả về ACK frame.
5. Peer A nhận ACK, cập nhật trạng thái “delivered”.

Luồng direct message và ACK



Hình 3.2: Sơ đồ luồng gửi tin nhắn trực tiếp (Direct Message)

3.2 Thiết kế các thành phần

Hệ thống P2P Chat được chia thành các thành phần độc lập, có trách nhiệm rõ ràng. Mỗi thành phần giao tiếp với nhau thông qua giao diện được định nghĩa sẵn, đảm bảo tính module hóa và khả năng mở rộng.

3.2.1. Bootstrap Server

Bootstrap Server là thành phần trung tâm phụ trợ của hệ thống P2P Chat, đóng vai trò hỗ trợ điều phối nhưng không tham gia trực tiếp vào luồng truyền tin nhắn chính giữa các peer. Toàn bộ tin nhắn chat được truyền trực tiếp qua kết nối TCP giữa các peer; Bootstrap Server chỉ cung cấp các dịch vụ nền tảng cần thiết để các peer có thể tìm thấy nhau và hoạt động ổn định.

Cụ thể, Bootstrap Server đảm nhận sáu nhiệm vụ cốt lõi sau:

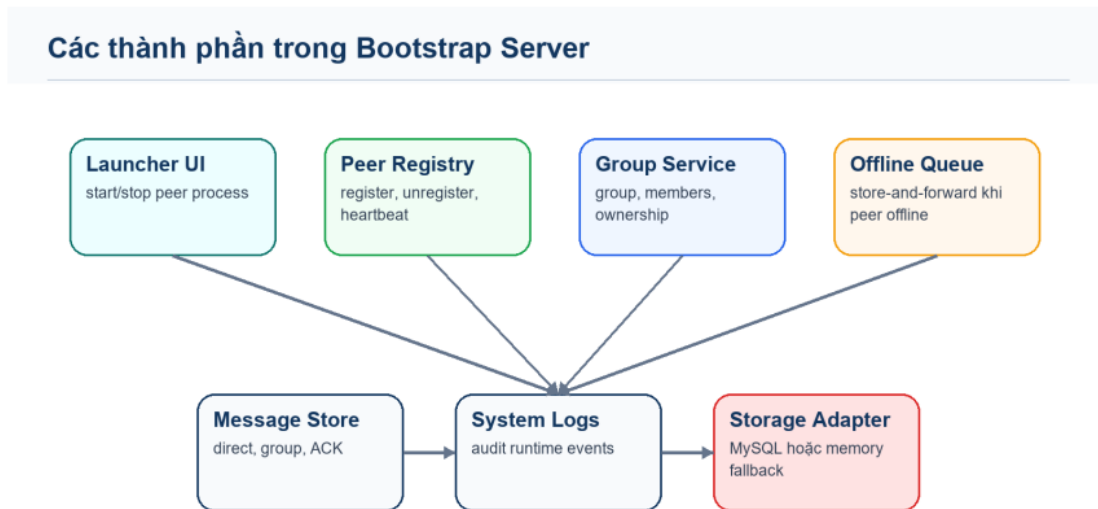
- Lưu trữ và cung cấp danh sách peer đang online phục vụ peer discovery.
- Quản lý trạng thái online/offline của từng peer thông qua cơ chế heartbeat định kỳ.
- Lưu trữ metadata nhóm chat, bao gồm thông tin group, danh sách thành viên và quyền sở hữu nhóm.
- Duy trì hàng đợi tin nhắn offline theo cơ chế store-and-forward, đảm bảo tin nhắn được chuyển đến peer đích khi peer đó kết nối trở lại.
- Ghi lịch sử tin nhắn, ACK, metadata file transfer và log hệ thống vào cơ sở

dữ liệu MySQL.

- Cung cấp giao diện Launcher UI cho phép quản trị viên khởi động hoặc dừng các peer process trực tiếp từ trình duyệt.

3.2.1.1 Cấu trúc thành phần

Bootstrap Server được tổ chức thành bảy module chức năng độc lập, mỗi module chịu trách nhiệm một nhóm nghiệp vụ riêng biệt, như minh họa trong Hình 3.2.



Hình 3.2: Các thành phần trong Bootstrap Server

Bốn module lớp trên xử lý các nghiệp vụ hướng peer:

- Launcher UI (launcher.js, views/index.ejs, public/app.js): Cung cấp giao diện web cho phép khởi động và dừng peer process con, phục vụ mục đích quản trị và thử nghiệm hệ thống.
- Peer Registry (routes.js): Xử lý toàn bộ vòng đời đăng ký của peer, bao gồm register khi khởi động, unregister khi tắt và heartbeat định kỳ để duy trì trạng thái online.
- Group Service (routes.js): Quản lý metadata nhóm chat gồm tạo nhóm, liệt kê nhóm, thêm/xóa thành viên và xác định quyền sở hữu nhóm.
- Offline Queue (routes.js): Lưu trữ tin nhắn gửi đến peer đang offline theo cơ chế store-and-forward; tin nhắn được giữ trong hàng đợi cho đến khi peer đích online và xác nhận nhận thành công (ACK).

Ba module lớp dưới xử lý lưu trữ và ghi nhận:

- Message Store (routes.js): Lưu lịch sử tin nhắn trực tiếp (direct), tin nhắn nhóm (group) và các bản ghi ACK vào cơ sở dữ liệu.
- System Logs (routes.js): Ghi nhận các sự kiện runtime của hệ thống phục vụ audit và giám sát.
- Storage Adapter (server.js): Lớp trừu tượng hóa việc lưu trữ dữ liệu, hỗ trợ MySQL làm backend chính và memory store làm fallback khi MySQL

không khả dụng.
Toàn bộ module được khởi tạo và kết nối trong server.js — điểm vào duy nhất của Bootstrap Server.

3.2.1.2. Các API endpoint chính

Toàn bộ giao tiếp giữa peer và Bootstrap Server được thực hiện qua REST API, được đăng ký tập trung trong routes.js. Bảng 3.1 liệt kê các endpoint chính theo nhóm chức năng.

Nhóm Peer Registry:

Endpoint	Method	Mô tả
/api/register	POST	Peer đăng ký địa chỉ và thông tin khi khởi động
/api/unregister	POST	Peer hủy đăng ký khi tắt
/api/heartbeat	POST	Peer gửi tín hiệu still-alive định kỳ
/api/peers	GET	Lấy danh sách toàn bộ peer đang online
/api/peers/:peerId	GET	Lấy thông tin chi tiết một peer cụ thể

Nhóm Group Service:

Endpoint	Method	Mô tả
/api/groups	POST/GET	Tạo mới và liệt kê danh sách nhóm
/api/groups/:id/members	POST	Thêm thành viên vào nhóm

Nhóm Offline Queue:

Endpoint	Method	Mô tả
/api/offline-messages	POST/GET	Lưu và lấy tin nhắn offline
/api/offline-messages/ack	POST	Xác nhận đã giao tin nhắn offline thành công

Nhóm Message Store & Logs:

Endpoint	Method	Mô tả
/api/messages/direct	POST	Lưu tin nhắn trực tiếp vào cơ sở dữ liệu
/api/messages/group	POST	Lưu tin nhắn nhóm vào cơ sở dữ liệu
/api/acks	POST	Lưu bản ghi ACK vào cơ sở dữ liệu
/api/file-transfers	POST	Lưu metadata của phiên truyền file
/api/logs	POST/GET	Ghi và đọc log hệ thống

Nhóm Launcher & Health:

Endpoint	Method	Mô tả
/api/launcher/start	POST	Khởi động một peer process mới
/api/launcher/stop	POST	Dừng một peer process đang chạy
/health	GET	Kiểm tra trạng thái hoạt động của server

3.2.1.3. Cơ chế theo dõi trạng thái peer (Heartbeat & TTL)

Bootstrap Server sử dụng cơ chế Heartbeat kết hợp TTL (Time-To-Live) để xác định trạng thái thực tế của từng peer mà không cần duy trì kết nối liên tục. Cơ chế này hoạt động theo ba bước:

Đầu tiên, mỗi peer chủ động gửi yêu cầu POST /api/heartbeat đến Bootstrap Server theo chu kỳ 8 giây một lần. Đây là tín hiệu xác nhận peer vẫn đang hoạt động bình thường.

Tiếp theo, mỗi khi nhận được heartbeat, Bootstrap Server cập nhật trường last_seen của peer tương ứng trong bảng peers của cơ sở dữ liệu, ghi nhận thời điểm hoạt động gần nhất.

Cuối cùng, khi có yêu cầu GET /api/peers, Bootstrap Server không trả về toàn bộ danh sách mà chỉ lọc và trả về những peer có last_seen trong vòng 30 giây trở lại (peerTtlMs = 30000). Mọi peer vượt quá ngưỡng TTL này — tức là không gửi heartbeat trong 30 giây — sẽ tự động bị coi là offline và bị loại khỏi kết quả trả về.

Cơ chế này đảm bảo danh sách peer luôn phản ánh trạng thái thực tế của

mạng mà không cần Bootstrap Server chủ động kiểm tra từng peer, giảm thiểu overhead và tăng khả năng mở rộng.

3.2.2. Peer Node

3.2.2.1. Vai trò

Mỗi Peer Node trong hệ thống P2P Chat vừa đóng vai trò client (khởi tạo kết nối và gửi tin nhắn đến peer khác) vừa đóng vai trò server (lắng nghe và xử lý tin nhắn đến từ các peer khác). Sau khi hoàn tất đăng ký với Bootstrap Server, peer hoạt động hoàn toàn tự chủ — mọi luồng truyền tin nhắn đều diễn ra trực tiếp qua TCP mà không cần sự can thiệp của Bootstrap Server.

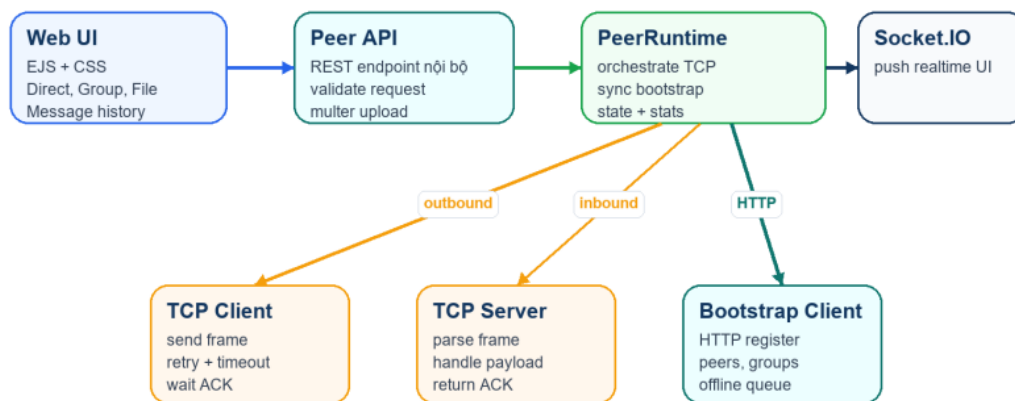
3.2.2.2. Cấu trúc thành phần

Peer Node được tổ chức thành tám module chức năng, phân tầng rõ ràng từ giao diện người dùng đến lớp truyền thông mạng, như minh họa trong Hình 3.3.

File	Vai trò
src/peer/server.js	Khởi tạo Express và Socket.IO, tích hợp PeerRuntime, phục vụ Web UI
src/peer/services/peerRuntime.js	Lớp điều phối trung tâm: quản lý vòng đời, gửi/nhận tin, group, file và churn simulation
src/peer/services/bootstrapClient.js	HTTP client giao tiếp với các REST API của Bootstrap Server
src/peer/tcp/client.js	Gửi payload qua TCP và chờ ACK phản hồi từ peer đích
src/peer/tcp/server.js	Lắng nghe kết nối TCP đến, phân tích payload và trả ACK
src/peer/routes/api.js	REST API nội bộ phục vụ các yêu cầu từ Web UI của peer
src/peer/views/index.ejs	Giao diện Web UI của peer, được render phía server
src/peer/public/app.js	Logic frontend của peer, kết nối realtime với server qua Socket.IO client

Kiến trúc phân tầng của Peer Node gồm bốn lớp chính. Lớp giao diện bao gồm Web UI (EJS + CSS) hiển thị lịch sử tin nhắn và các chức năng Direct, Group, File. Lớp API nội bộ gồm Peer API xử lý và xác thực yêu cầu từ Web UI, hỗ trợ upload file qua multer. Lớp điều phối trung tâm là PeerRuntime, chịu trách nhiệm orchestrate toàn bộ hoạt động TCP, đồng bộ với Bootstrap và duy trì trạng thái nội bộ; kết hợp với Socket.IO để đẩy cập nhật realtime lên giao diện người dùng. Lớp mạng gồm TCP Client (gửi frame, retry, chờ ACK), TCP Server (nhận frame, xử lý payload, trả ACK) và Bootstrap Client (HTTP register, đồng bộ peers, chờ ACK).

Các thành phần trong Peer App



Hình 3.3: Các thành phần trong Peer App

3.2.2.3. Vòng đời của một Peer Node

Khi khởi động, Peer Node thực hiện tuần tự các bước khởi tạo sau:

Đầu tiên, Express Server và Socket.IO được khởi động để phục vụ Web UI và kênh realtime cho người dùng. Tiếp theo, PeerRuntime.start() được gọi để thực hiện toàn bộ quy trình khởi tạo mạng: mở TCP Server lắng nghe trên cổng TCP được cấu hình; đăng ký thông tin peer với Bootstrap Server qua POST /api/register; đồng bộ danh sách peer, nhóm và lịch sử tin nhắn từ Bootstrap; nhận và giao toàn bộ tin nhắn offline đang chờ trong hàng đợi.

Sau khi hoàn tất khởi tạo, peer duy trì bốn vòng lặp định kỳ chạy song song:

Tác vụ	Chu kỳ	Mục đích
Heartbeat	8 giây	Duy trì trạng thái online trên Bootstrap Server
Sync peers	5 giây	Cập nhật danh sách peer đang online

Sync groups	5 giây	Đồng bộ metadata nhóm chat
Poll offline queue	5 giây	Nhận tin nhắn offline tích lũy trong thời gian mất kết nối

3.2.2.4. Lớp *PeerRuntime* — Điều phối trung tâm

PeerRuntime là module quan trọng nhất của *Peer Node*, đóng vai trò bộ điều phối (orchestrator) toàn bộ hoạt động của peer. Mọi luồng xử lý — từ gửi tin nhắn, nhận tin, quản lý nhóm đến truyền file — đều được điều phối qua lớp này.

Trạng thái nội bộ

PeerRuntime duy trì một tập hợp trạng thái nội bộ phản ánh toàn bộ ngữ cảnh hoạt động của peer tại mọi thời điểm:

Thuộc tính	Kiểu	Ý nghĩa
peers	Array	Danh sách peer đã biết, đồng bộ từ Bootstrap Server
groups	Array	Danh sách nhóm chat mà peer đang tham gia
messages	Array	Lịch sử tin nhắn, giới hạn tối đa 300 bản ghi
logs	Array	Log runtime nội bộ, giới hạn tối đa 150 bản ghi
stats	Object	Thống kê truyền tin: sent, delivered, failed, received, queued
seenMessageIds	Set	Bộ đệm chống tin nhắn trùng lặp, giới hạn tối đa 1.000 ID
churnOffline	Boolean	Cờ kích hoạt chế độ offline mô phỏng phục vụ kiểm thử churn

Các phương thức gửi tin

- **PeerRuntime** cung cấp bốn phương thức gửi tin tương ứng với bốn kịch bản truyền thông khác nhau:
- **sendDirect(toPeerId, content)** thực hiện gửi tin nhắn trực tiếp 1-1 đến peer đích qua kết nối TCP. Nếu kết nối TCP thất bại, hệ thống tự động thử chuyển tiếp qua peer trung gian (relay). Trong trường hợp relay cũng thất bại, tin nhắn được đưa vào hàng đợi offline trên Bootstrap Server để giao sau khi peer đích online trở lại.

- **sendGroup(groupId, members, content)** gửi tin nhắn lần lượt đến từng thành viên trong nhóm theo cơ chế unicast tuần tự. Kết quả gửi đến từng peer được theo dõi riêng biệt và phân loại thành ba trạng thái: delivered (giao thành công), partial (một phần thành viên nhận được) hoặc failed_queued (thất bại và đã đưa vào hàng đợi offline).
- **broadcast(content)** gửi tin nhắn đến toàn bộ peer đang online tại thời điểm gửi. Phương thức này không hỗ trợ offline queue — những peer đang offline tại thời điểm broadcast sẽ không nhận được tin nhắn này.
- **sendFile(toPeerId, file)** mã hóa file thành chuỗi Base64 và truyền qua TCP payload đến peer đích. Khác với sendDirect, phương thức này chỉ thực hiện khi peer đích đang online và không hỗ trợ đưa vào offline queue do kích thước payload lớn.

3.3. Thiết kế cơ sở dữ liệu

Hệ thống P2P Chat sử dụng MySQL làm hệ quản trị cơ sở dữ liệu chính, kết hợp với cơ chế fallback về bộ nhớ RAM khi MySQL không khả dụng. Cơ sở dữ liệu không tham gia vào luồng truyền tin nhắn chính — tin nhắn được truyền trực tiếp qua TCP giữa các peer — mà chỉ đóng vai trò lưu trữ bền vững cho trạng thái peer, lịch sử tin nhắn, hàng đợi offline, ACK và log hệ thống.

3.3.1 Hệ quản trị cơ sở dữ liệu

Hệ thống được cấu hình với MySQL 8.x, sử dụng bộ ký tự utf8mb4 và collation utf8mb4_unicode_ci để hỗ trợ đầy đủ Unicode, bao gồm emoji và ký tự đặc biệt. Kết nối được quản lý qua thư viện mysql2/promise của Node.js với connection pool tối đa 10 kết nối đồng thời, đảm bảo hiệu năng khi nhiều peer thao tác cơ sở dữ liệu cùng lúc.

3.3.2. Chiến lược lưu trữ kép (Dual Storage Strategy)

Hệ thống hỗ trợ hai chế độ lưu trữ thông qua lớp trừu tượng Store, được lựa chọn tự động tại thời điểm khởi động dựa trên biến môi trường và khả năng kết nối thực tế.

Chế độ	Lớp triển khai	Đặc điểm
MySQL	MySqlStore	Lưu trữ bền vững, dữ liệu tồn tại sau khi restart
In-Memory	MemoryStore	Lưu trong RAM, mất khi tắt server, phù hợp demo nhanh

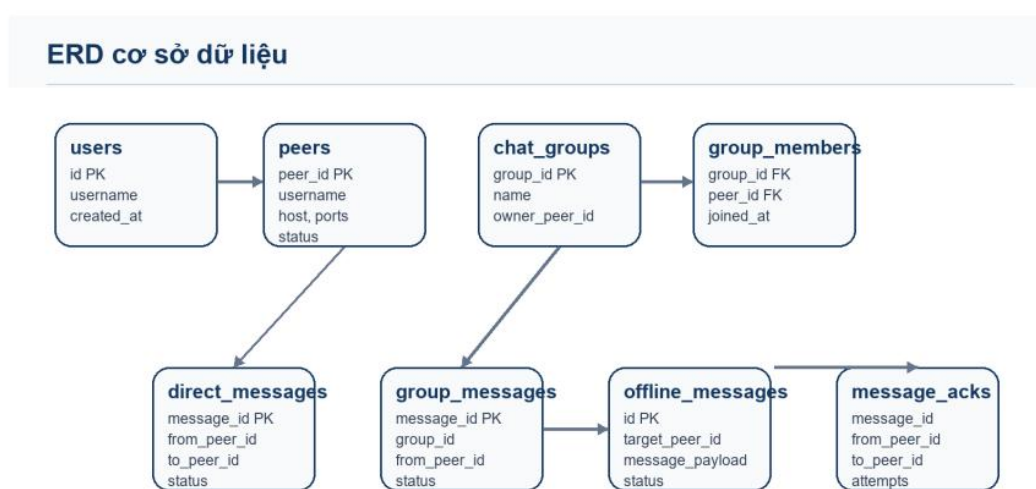
Quy trình quyết định chế độ lưu trữ diễn ra tuần tự: nếu DB_ENABLED=false, hệ thống khởi tạo MemoryStore ngay lập tức mà không thử kết nối MySQL. Nếu DB_ENABLED=true, hệ thống thử kết nối MySQL;

thành công thì dùng MySQLStore, thất bại thì kiểm tra tiếp DB_FALLBACK_MEMORY — nếu bằng true thì chuyển sang MemoryStore, nếu false thì server dừng hoàn toàn và ném lỗi.

Cả hai lớp MySQLStore và MemoryStore đều triển khai cùng một giao diện đồng nhất gồm 20 phương thức, đảm bảo toàn bộ mã nguồn ứng dụng không cần thay đổi khi chuyển đổi giữa hai chế độ.

3.3.3. Sơ đồ quan hệ thực thể (ER Diagram)

Cơ sở dữ liệu gồm 10 bảng, được tổ chức xoay quanh bảng trung tâm peers. Hình 3.5 thể hiện sơ đồ quan hệ giữa 8 bảng nghiệp vụ chính.



Hình 3.5: Sơ đồ ERD (Entity Relationship Diagram) của cơ sở dữ liệu

3.3.4. Mô tả chi tiết từng bảng

Bảng users — Thông tin người dùng

Lưu thông tin tài khoản người dùng. Mỗi peer khi đăng ký sẽ tự động tạo bản ghi user tương ứng. Khóa chính id tự tăng, trường username là duy nhất trong toàn hệ thống.

Bảng peers — Registry thông tin peer

Là bảng trung tâm của toàn bộ hệ thống, lưu thông tin đăng ký và trạng thái của mỗi peer node. Khóa chính peer_id là định danh chuỗi do peer tự đặt (ví dụ: peer-alice). Bảng có hai chỉ mục phụ: idx_peers_status(status) để lọc nhanh peer online và idx_peers_username(username) để tra cứu theo tên người dùng. Khi peer đăng ký, hệ thống sử dụng INSERT ... ON DUPLICATE KEY UPDATE để cập nhật thông tin mà không gây lỗi khi peer restart.

Bảng peer_status_logs — Lịch sử trạng thái peer

Ghi nhận toàn bộ lịch sử chuyển đổi trạng thái online/offline của từng peer, phục vụ mục đích phân tích và debug sau sự cố. Chỉ mục idx_peer_status_logs_peer(peer_id) hỗ trợ truy vấn nhanh theo peer.

Bảng chat_groups — Metadata nhóm chat

Lưu thông tin nhóm chat. Mỗi nhóm có một peer làm chủ sở hữu (owner_peer_id), định danh nhóm là UUID sinh tự động. Chỉ mục idx_groups_owner(owner_peer_id) hỗ trợ tìm nhanh các nhóm do một peer cụ thể sở hữu.

Bảng group_members — Thành viên nhóm

Bảng liên kết N-N giữa chat_groups và peers. Khóa chính composite (group_id, peer_id) đảm bảo mỗi peer chỉ xuất hiện một lần trong mỗi nhóm. Truy vấn đặc trưng dùng GROUP_CONCAT để lấy danh sách nhóm kèm toàn bộ thành viên trong một lần truy vấn duy nhất, tránh vấn đề N+1 query.

Bảng direct_messages — Tin nhắn trực tiếp

Lưu lịch sử tin nhắn 1-1 giữa hai peer, hỗ trợ cả bản rõ (content) và bản mã hóa (encryption_payload dạng JSON chứa algorithm, iv, tag và data). Trạng thái tin nhắn gồm bốn giá trị: sending, delivered, failed_queued và received. Cơ chế upsert cho phép cập nhật trạng thái từ sending sang delivered mà không cần truy vấn kiểm tra trước.

Bảng group_messages — Tin nhắn nhóm

Cấu trúc tương tự direct_messages, thay trường to_peer_id bằng group_id. Khi lấy tin nhắn nhóm cho một peer, hệ thống thực hiện JOIN với bảng group_members để chỉ trả về tin nhắn thuộc các nhóm mà peer đó tham gia.

Bảng offline_messages — Hàng đợi tin nhắn offline

Lưu các tin nhắn chờ giao theo cơ chế store-and-forward. Vòng đời một bản ghi gồm ba bước: khi gửi TCP thất bại, tin nhắn được INSERT với status='pending'; khi peer đích online và poll hàng đợi, tin nhắn được trả về và xử lý; sau khi xác nhận thành công, bản ghi được UPDATE thành status='delivered' kèm delivered_at. Chỉ mục composite idx_offline_target_status(target_peer_id, status) đảm bảo truy vấn poll có hiệu năng cao ngay cả khi hàng đợi lớn.

Bảng message_acks — Xác nhận giao tin

Khóa chính composite (message_id, from_peer_id, to_peer_id) cho phép theo dõi kết quả phân phối của từng tin nhắn theo từng cặp gửi-nhận riêng biệt. Trường status ghi nhận bốn kết quả: delivered (TCP thành công), delivered_via_relay (qua peer trung gian), queued_offline (peer offline, đưa vào hàng đợi) và failed (thất bại hoàn toàn). Trường attempts ghi số lần thử gửi tích lũy.

Bảng file_transfers — Metadata truyền file

Lưu metadata của mỗi phiên truyền file gồm tên file, MIME type, kích thước và đường dẫn lưu trên hệ thống file tại thư mục received_files/<peer_id>/. Nội dung file không được lưu trong cơ sở dữ liệu — chỉ có đường dẫn tham chiếu

được ghi tại trường `saved_path`.

Bảng `system_logs` — Log hệ thống

Ghi nhận toàn bộ sự kiện runtime với trường `scope` phân loại nguồn gốc (`bootstrap`, `launcher`, `system`, `peer`) và trường `meta_payload` dạng JSON cho phép đính kèm dữ liệu cấu trúc tùy ý. Chỉ mục `idx_system_logs_created(created_at)` hỗ trợ phân trang theo thời gian hiệu quả.

3.3.5. Tổng hợp quan hệ giữa các bảng

Bảng nguồn	Bảng đích	Loại quan hệ	Mô tả
users	peers	1 — N	Một user có thể sở hữu nhiều peer
peers	group_members	1 — N	Một peer tham gia nhiều nhóm
chat_groups	group_members	1 — N	Một nhóm có nhiều thành viên
peers	direct_messages	1 — N	Peer gửi/nhận nhiều tin nhắn trực tiếp
chat_groups	group_messages	1 — N	Nhóm chứa nhiều tin nhắn
peers	offline_messages	1 — N	Peer có nhiều tin nhắn offline đang chờ
direct_messages	message_acks	1 — N	Mỗi tin nhắn có nhiều ACK theo từng cặp gửi-nhận
peers	file_transfers	1 — N	Peer gửi/nhận nhiều file

3.4. Thiết kế Giao thức và các Luồng xử lý chính

Trong hệ thống phân tán P2P, việc thiết kế giao thức giao tiếp là yếu tố sống còn để các nút (peer) có thể hiểu và làm việc với nhau mà không cần sự can thiệp liên tục của server trung tâm. Hệ thống sử dụng giao thức dựa trên TCP với định dạng dữ liệu JSON.

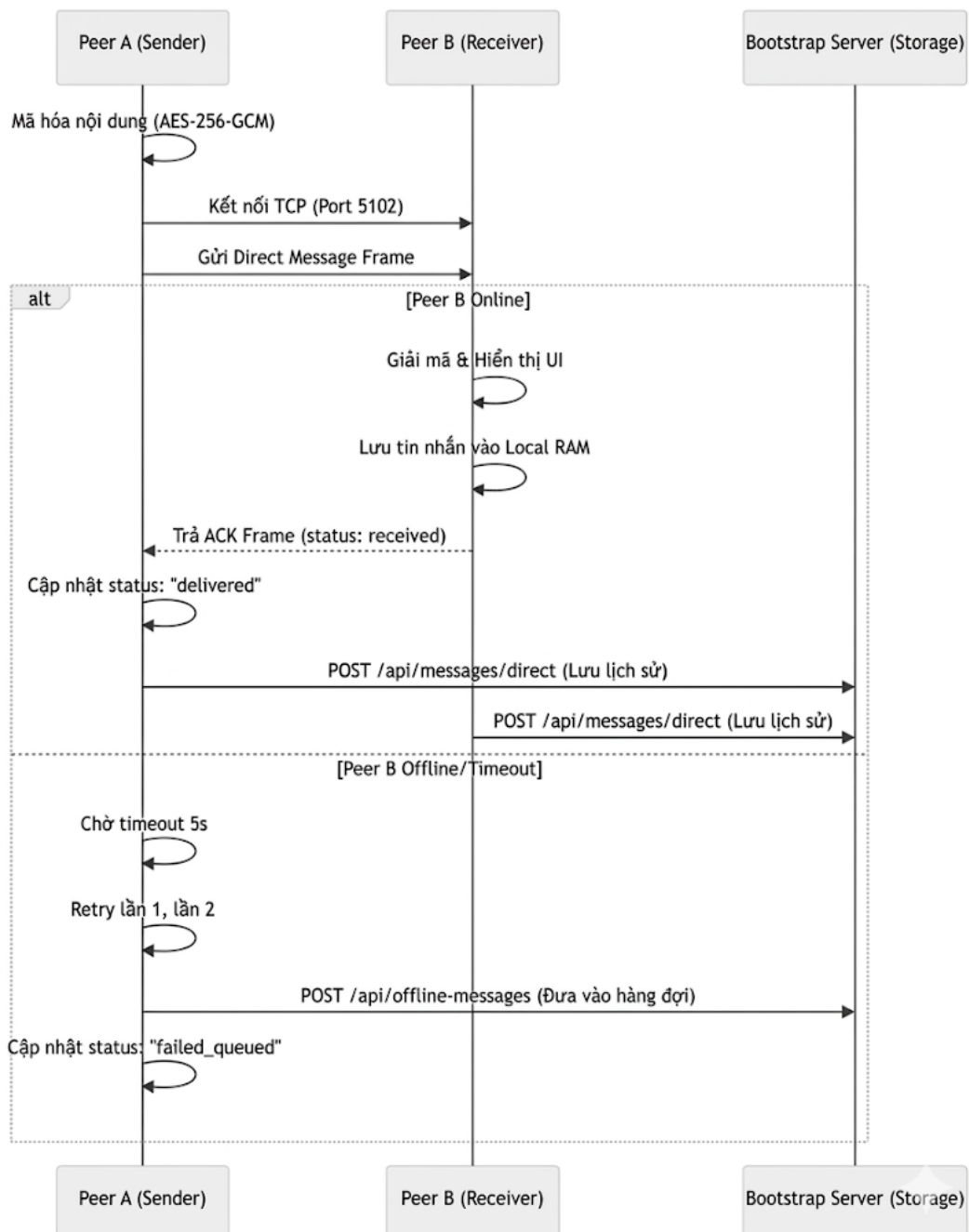
3.4.1. Giao thức truyền tin (Wire Format)

Hệ thống sử dụng kỹ thuật Line-Delimited JSON qua kết nối TCP. Mỗi "gói tin" (frame) là một đối tượng JSON được mã hóa và kết thúc bằng ký tự xuống dòng (`\n`).

- Ưu điểm: Dễ debug, tương thích tốt với các ngôn ngữ lập trình, xử lý được dữ liệu có kích thước thay đổi.
- Cơ chế đóng gói (Framing):
 - Gửi: `JSON.stringify(payload) + '\n'`
 - Nhận: Tích lũy vào buffer, cắt theo ký tự `\n`, sau đó `JSON.parse()`.

3.4.2. Luồng gửi tin nhắn trực tiếp (Direct Message)

Đây là luồng xử lý cốt lõi, tin nhắn đi trực tiếp giữa hai peer.
Sơ đồ trình tự (Sequence Diagram):



3.4.3. Cơ chế ACK và Retry

Để đảm bảo tính tin cậy của quá trình truyền dữ liệu trên mô hình giao tiếp P2P, hệ thống triển khai cơ chế xác nhận và gửi lại gói tin khi xảy ra lỗi truyền tải. Các cơ chế chính bao gồm:

1. ACK (Acknowledgment):

Mỗi gói tin được gửi đi đều yêu cầu phía nhận phản hồi một gói ACK chứa cùng messageId. Khi nhận được ACK hợp lệ, phía gửi xác nhận rằng dữ liệu đã được truyền thành công và dừng quá trình chờ phản hồi.

2. Retry (Thử gửi lại):

Nếu sau khoảng thời gian 5 giây phía gửi không nhận được ACK tương ứng, hệ thống sẽ xem kết nối hiện tại là thất bại, tiến hành đóng socket và thực hiện gửi lại gói tin.

3. Exponential Backoff:

Để hạn chế tình trạng nghẽn mạng khi nút nhận đang quá tải hoặc mất kết nối tạm thời, thời gian chờ giữa các lần retry sẽ tăng dần theo cấp số nhân, ví dụ:

200ms → 400ms → 800ms → ...

Cơ chế này giúp giảm số lượng kết nối được tạo liên tục trong thời gian ngắn.

4. Deduplication (Chống trùng lặp):

Trong trường hợp ACK bị thất lạc, phía gửi có thể retransmit cùng một gói tin nhiều lần. Để tránh xử lý dữ liệu lặp lại, Peer nhận sử dụng cấu trúc Set(seenMessageIds) để lưu các messageId đã xử lý. Nếu nhận được gói tin có messageId đã tồn tại, hệ thống sẽ bỏ qua nội dung nhưng vẫn phản hồi ACK để đồng bộ trạng thái với phía gửi.

3.4.4. Luồng xử lý tin nhắn Offline (Store-and-Forward)

Để đảm bảo khả năng liên lạc ngay cả khi một Peer tạm thời mất kết nối, hệ thống triển khai cơ chế Store-and-Forward thông qua Bootstrap Server. Cơ chế này cho phép lưu trữ tạm thời các tin nhắn chưa gửi được và chuyển tiếp lại khi Peer đích hoạt động trở lại.

Quy trình xử lý bao gồm các bước sau:

1. Phát hiện Peer Offline:

Peer A thực hiện gửi tin nhắn trực tiếp đến Peer B thông qua kết nối TCP. Nếu toàn bộ số lần retry đều thất bại hoặc không nhận được ACK hợp lệ, hệ thống xác định Peer B đang offline hoặc không thể kết nối.

2. Lưu trữ tại Bootstrap Server:

Sau khi gửi trực tiếp thất bại, Peer A sẽ gọi API:

POST /api/offline-messages

để gửi payload chứa nội dung tin nhắn đến Bootstrap Server. Server tiếp nhận và lưu thông tin vào bảng offline_messages với trạng thái ban đầu là pending.

3. Cơ chế Polling:

Khi Peer B online trở lại, PeerRuntime sẽ khởi động tiến trình kiểm tra tin nhắn offline theo chu kỳ thông qua hàm pollOfflineMessages. Tiến trình

này thực hiện polling định kỳ mỗi 5 giây bằng cách gửi request đến Server để truy vấn các tin nhắn chưa nhận.

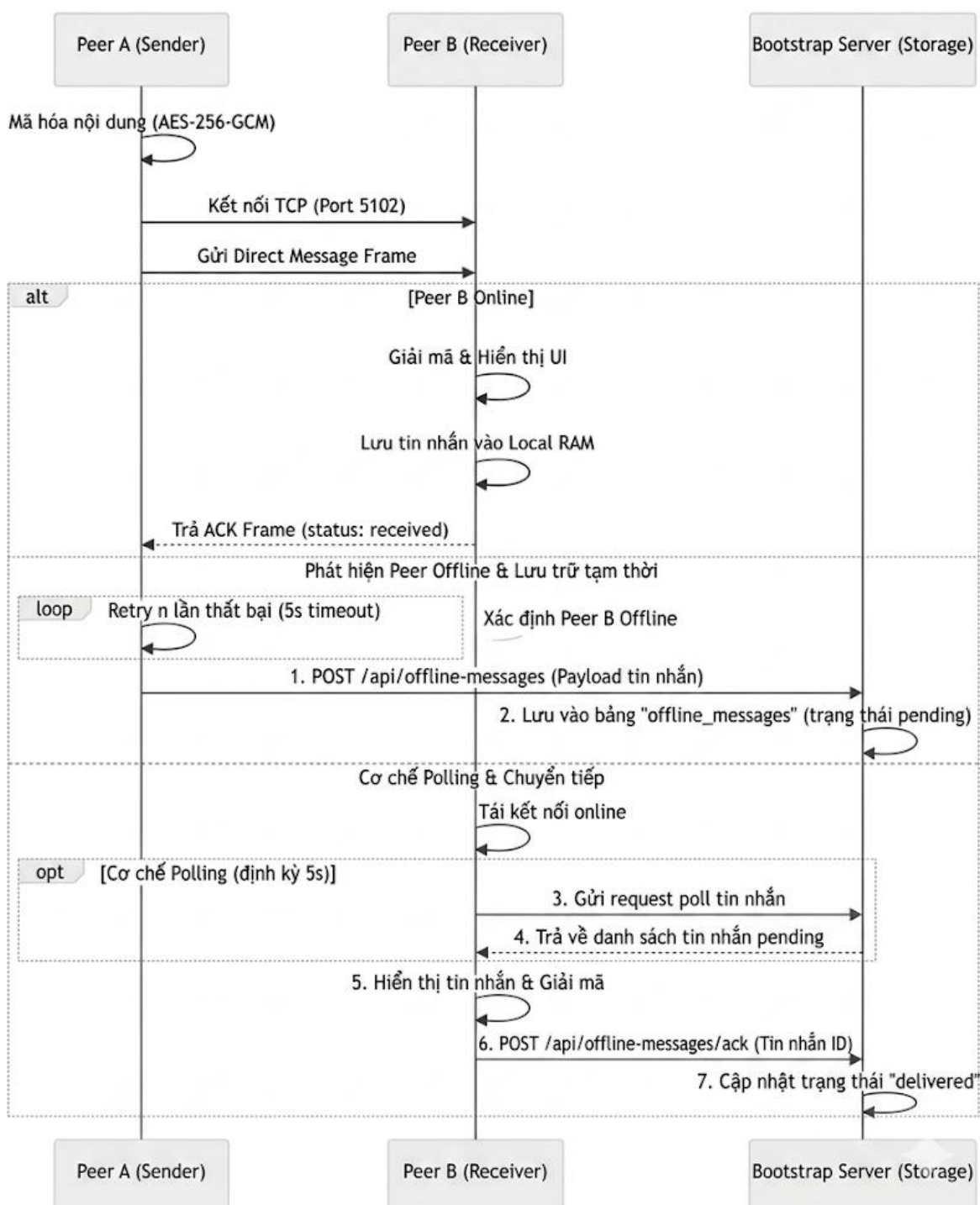
4. Chuyển tiếp và xác nhận:

Bootstrap Server trả về danh sách các tin nhắn có trạng thái pending dành cho Peer B. Sau khi Peer B nhận và hiển thị thành công nội dung tin nhắn, hệ thống gửi request:

POST/api/offline-messages/ack

nhằm xác nhận việc nhận dữ liệu. Server sau đó cập nhật trạng thái tin nhắn sang delivered để tránh gửi lại trong các lần polling tiếp theo.

Cơ chế Store-and-Forward giúp hệ thống duy trì khả năng trao đổi dữ liệu liên tục, đồng thời tăng tính tin cậy trong môi trường mạng phân tán nơi các Peer có thể online hoặc offline bất kỳ lúc nào.



3.4.5. Luồng Relay tin nhắn (Trung gian)

Trong một số trường hợp, kết nối TCP trực tiếp giữa hai Peer không thể thiết lập do ảnh hưởng của NAT, Firewall hoặc các chính sách mạng nội bộ. Khi đó, mặc dù Peer B vẫn đang online theo thông tin từ Bootstrap Server, Peer A vẫn không thể gửi dữ liệu trực tiếp. Để xử lý tình huống này, hệ thống triển khai cơ chế Relay Message thông qua một Peer trung gian.

Quy trình hoạt động như sau:

1. Tìm Peer trung gian:

Sau khi kết nối trực tiếp đến Peer B thất bại, Peer A gửi yêu cầu đến Bootstrap Server để lấy danh sách các Peer đang online. Từ danh sách này, hệ thống lựa chọn một Peer trung gian phù hợp, ví dụ Peer C.

2. Đóng gói gói tin Relay:

Peer A tạo một gói tin có kiểu `relay_message`. Bên trong gói tin chứa:

- a. Thông tin Peer đích (Peer B)
- b. `messageId`
- c. Nội dung tin nhắn gốc (`innerPayload`)
- d. Metadata phục vụ định tuyến và ACK

Gói tin này đóng vai trò như một “lớp vỏ” bao quanh dữ liệu gốc cần chuyển tiếp.

3. Chuyển tiếp qua Peer trung gian:

Peer A gửi gói `relay_message` cho Peer C với ý nghĩa:

“Hãy chuyển tiếp tin nhắn này đến Peer B giúp tôi.”

4. Relay đến Peer đích:

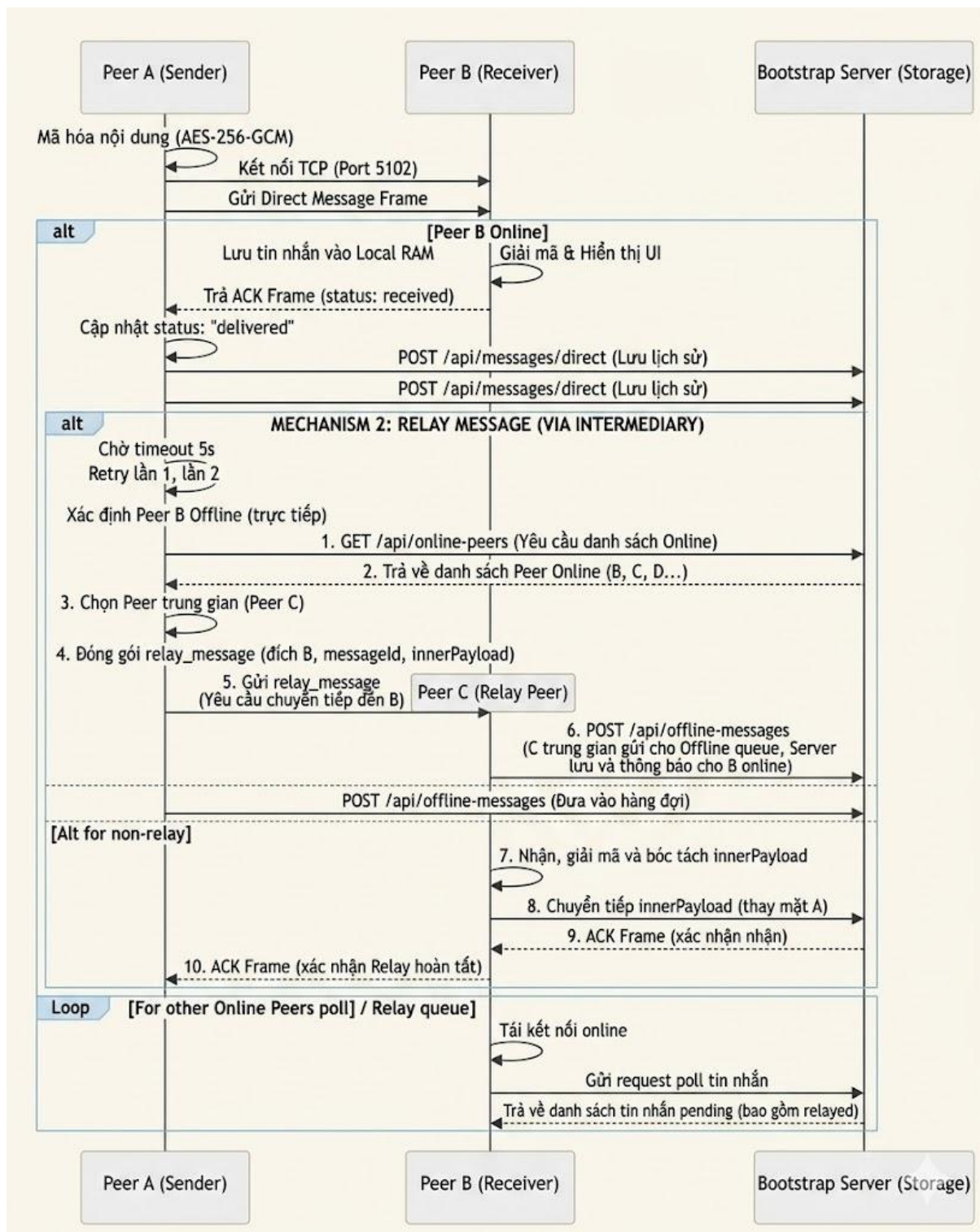
Sau khi nhận được gói relay, Peer C sẽ:

- a. Kiểm tra tính hợp lệ của payload
- b. Bóc tách phần `innerPayload`
- c. Thiết lập kết nối TCP đến Peer B
- d. Gửi lại nội dung gốc thay mặt Peer A

5. Cơ chế ACK hai tầng:

Khi Peer B nhận thành công dữ liệu, Peer B gửi ACK cho Peer C. Sau đó Peer C tiếp tục gửi ACK về lại cho Peer A để xác nhận rằng quá trình relay đã hoàn tất thành công.

Cơ chế Relay giúp cải thiện khả năng kết nối trong môi trường mạng phức tạp, đặc biệt đối với các Peer nằm sau NAT hoặc Firewall, đồng thời đảm bảo dữ liệu vẫn được truyền tải mà không phụ thuộc hoàn toàn vào kết nối trực tiếp giữa hai đầu cuối.



3.4.6. Luồng truyền tải File (TCP File Transfer)

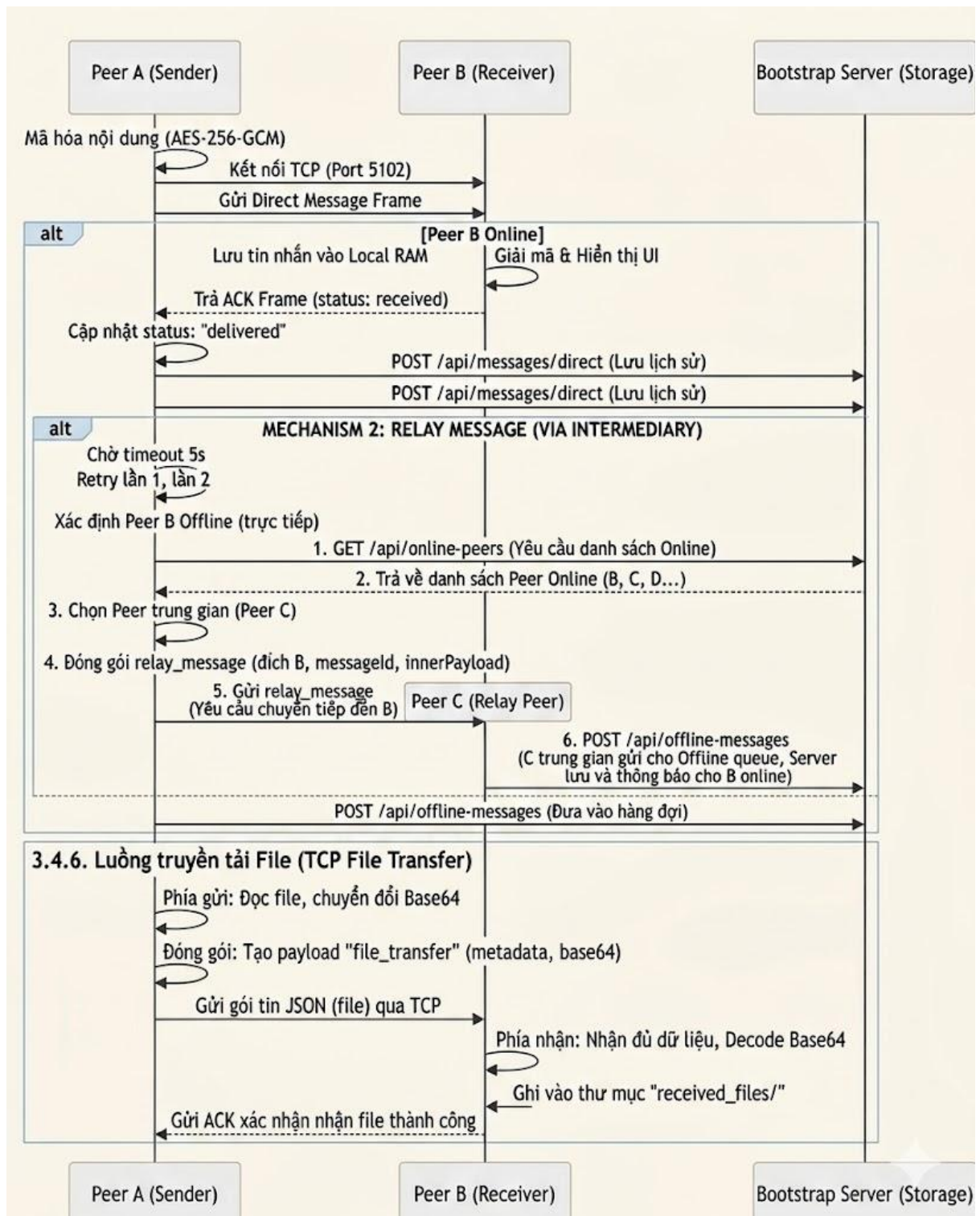
Vì TCP là luồng byte (byte-stream), hệ thống xử lý truyền file nhỏ bằng cách:

1. Phía gửi: Đọc file, chuyển đổi sang chuỗi Base64.
2. Đóng gói: Tạo payload file_transfer chứa metadata (tên file, size, mime) và chuỗi base64 của file.

3. Truyền tải: Gửi gói tin JSON khổng lồ này qua TCP.

4. Phía nhận:

- Nhận đủ dữ liệu JSON.
- Decode Base64 trở lại Buffer.
- Ghi vào thư mục `received_files/`.
- Gửi ACK xác nhận đã nhận file thành công.



3.4.7. Luồng Discovery và Heartbeat

Để duy trì trạng thái hoạt động của mạng P2P và đảm bảo các Peer luôn cập nhật được danh sách nút đang online, hệ thống triển khai cơ chế Discovery và Heartbeat. Cơ chế này giúp mạng tự động phát hiện các Peer mới tham gia, đồng thời loại bỏ các Peer đã mất kết nối hoặc ngừng hoạt động.

Quy trình hoạt động gồm các thành phần sau:

1. Register (Đăng ký Peer):

Khi một Peer khởi chạy, PeerRuntime sẽ gửi thông tin định danh lên Bootstrap Server thông qua API đăng ký. Thông tin bao gồm:

- a. peerId
- b. host
- c. tcpPort
- d. webPort
- e. thời gian khởi tạo kết nối

Server sau đó lưu Peer vào danh sách các nút đang hoạt động trong mạng.

2. Heartbeat (Duy trì trạng thái online):

Sau khi đăng ký thành công, mỗi Peer sẽ định kỳ gửi một gói heartbeat nhỏ (Ping) lên Bootstrap Server mỗi 8 giây nhằm thông báo rằng Peer vẫn đang hoạt động bình thường.

Gói heartbeat có kích thước rất nhỏ, chỉ chứa:

- a. peerId
- b. timestamp hiện tại

Server cập nhật trường lastSeen tương ứng với Peer gửi heartbeat.

3. Discovery (Khám phá Peer):

Để đồng bộ danh sách các nút trong mạng, mỗi Peer sẽ định kỳ gửi request: *GET /api/peers*

mỗi 5 giây để lấy danh sách các Peer hiện đang online. Thông qua cơ chế này, hệ thống có thể:

- a. Phát hiện Peer mới tham gia mạng
- b. Cập nhật trạng thái Peer vừa online
- c. Loại bỏ các Peer đã offline
- d. Hỗ trợ thiết lập kết nối trực tiếp hoặc relay

4. Auto-Cleanup (Tự động dọn dẹp):

Bootstrap Server liên tục kiểm tra thời gian heartbeat cuối cùng (lastSeen) của từng Peer. Nếu quá 30 giây không nhận được heartbeat từ một Peer, Server sẽ tự động chuyển trạng thái Peer đó sang offline.

Cơ chế này giúp:

- a. Tránh lưu các kết nối “ma”
- b. Đảm bảo danh sách Peer luôn chính xác
- c. Giảm lỗi kết nối đến các nút không còn hoạt động

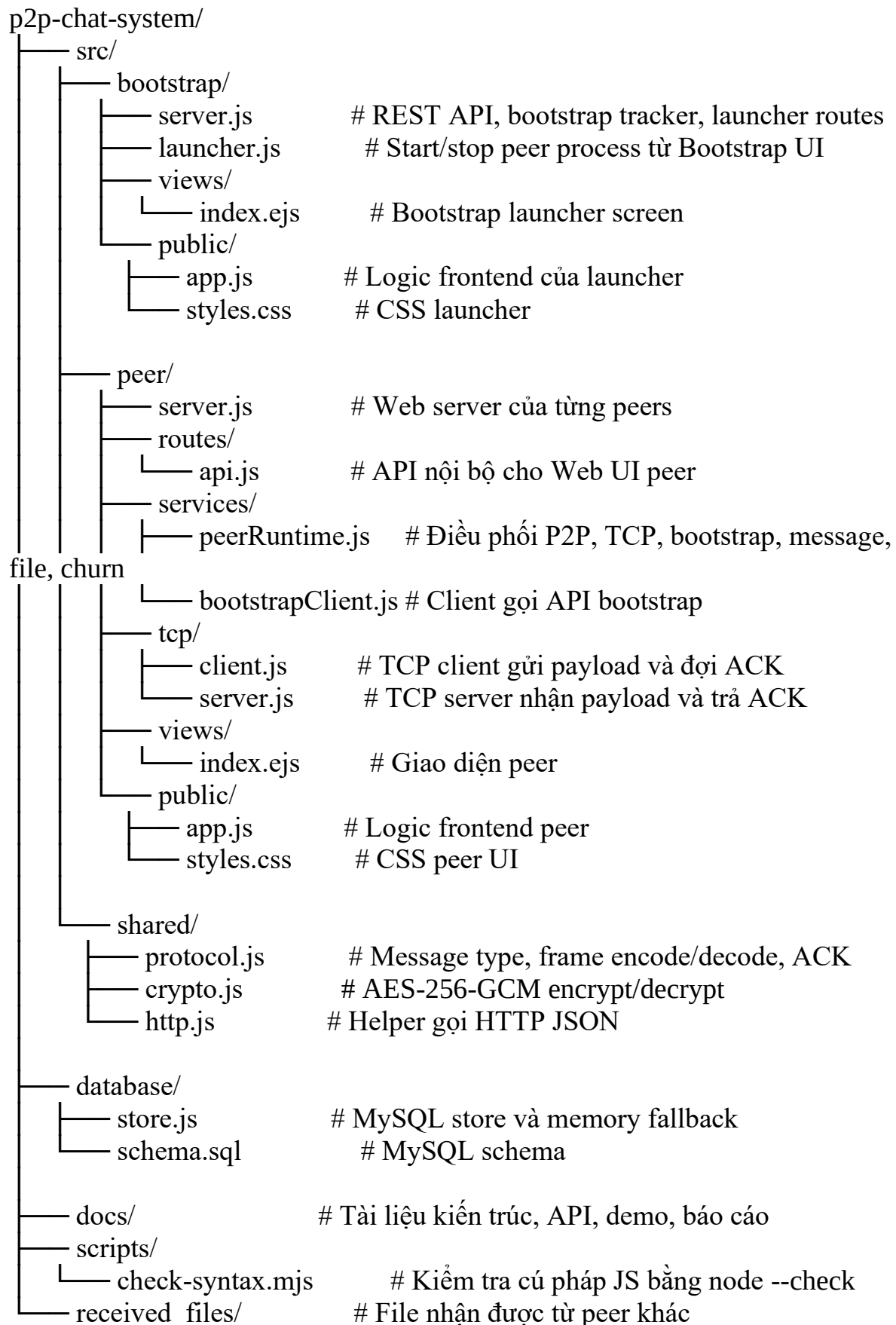
Nhờ cơ chế Discovery và Heartbeat, mạng P2P có khả năng tự thích nghi với sự thay đổi trạng thái của các nút trong thời gian thực, giúp duy trì tính ổn định và khả năng mở rộng của toàn hệ thống

CHƯƠNG IV : CÀI ĐẶT VÀ TRIỂN KHAI

4.1. Công nghệ sử dụng

Thành phần	Công nghệ sử dụng	Vai trò trong hệ thống
Runtime Environment	Node.js 18+	Cung cấp môi trường thực thi cho Bootstrap Server và các tiến trình Peer trong mạng P2P
Web Backend	Express 5.x	Xây dựng REST API, xử lý request/response và phục vụ giao diện Web UI
Realtime Communication	Socket.IO 4.x	Đồng bộ trạng thái thời gian thực, truyền log và cập nhật tin nhắn trực tiếp lên trình duyệt
P2P Networking	node:net (TCP Socket)	Thiết lập kết nối TCP và truyền dữ liệu trực tiếp giữa các Peer
Database Management	MySQL 8.x	Lưu trữ thông tin Peer, nhóm chat, tin nhắn, ACK và hàng đợi offline message
Template Engine	EJS	Render giao diện phía server theo mô hình server-side rendering
File Upload Handling	Multer 2.x	Tiếp nhận và xử lý file upload từ giao diện Web UI
Encryption	AES-256-GCM	Mã hóa nội dung tin nhắn và dữ liệu truyền qua kết nối TCP nhằm đảm bảo tính bảo mật
Configuration Management	dotenv	Đọc và quản lý các biến môi trường từ file .env
Unique Identifier Generator	uuid v13	Sinh định danh duy nhất cho message, transfer và các transaction trong hệ thống

4.2. Cấu trúc mã nguồn



4.3. Cấu hình môi trường và cơ sở dữ liệu

Hệ thống sử dụng tệp cấu hình biến môi trường .env nằm ở thư mục gốc để quản lý toàn bộ các thông số liên quan đến mạng, cơ sở dữ liệu và bảo mật.

4.3.1. Chi tiết các biến môi trường cấu hình

Khi khởi chạy hệ thống, file .env sẽ được nạp qua thư viện dotenv để cấu hình các thông số sau:

Tên biến môi trường	Giá trị mặc định	Giải thích chi tiết
BOOTSTRAP_PORT	3000	Cổng HTTP mà Bootstrap Server sử dụng để lắng nghe và tiếp nhận các kết nối từ peer.
BOOTSTRAP_URL	http://127.0.0.1:3000	Địa chỉ URL của Bootstrap Server để các peer gửi yêu cầu đăng ký và heartbeat.
PEER_ID	peer-a	Định danh duy nhất (Unique ID) của peer trong mạng P2P.
USERNAME	Alice	Tên hiển thị của peer trong hệ thống chat.
PEER_HOST	127.0.0.1	Địa chỉ IP hoặc hostname của peer để các peer khác có thể kết nối qua TCP socket.
TCP_PORT	5101	Cổng TCP mà peer mở để nhận và truyền dữ liệu trực tiếp giữa các peer.
WEB_PORT	3101	Cổng chạy giao diện Web UI để người dùng truy cập bằng trình duyệt.
DB_ENABLED	true	Bật (true) hoặc tắt (false) chức năng kết nối tới cơ sở dữ liệu MySQL.
DB_FALLBACK_MEMORY	true	Cho phép hệ thống lưu trữ tạm trên RAM khi MySQL gặp lỗi kết nối.
ENCRYPTION_ENABLED	true	Kích hoạt cơ chế mã hóa AES-256-GCM cho dữ liệu trước khi truyền

		qua mạng TCP.
P2P_SHARED_SECRET	(chuỗi bảo mật)	Khóa bí mật dùng chung giữa các peer để thực hiện mã hóa và giải mã dữ liệu.
MYSQL_HOST	127.0.0.1	Địa chỉ IP của máy chủ MySQL Server.
MYSQL_PORT	3306	Cổng dịch vụ của MySQL Server.
MYSQL_USER	root	Tài khoản dùng để đăng nhập vào MySQL.
MYSQL_PASSWORD	(mật khẩu của bạn)	Mật khẩu của tài khoản MySQL.
MYSQL_DATABASE	p2p_chat	Tên cơ sở dữ liệu dùng để lưu trữ dữ liệu của hệ thống chat P2P.

4.3.2. Cấu trúc Cơ sở dữ liệu (MySQL Database Schema)

Hệ thống lưu trữ bền vững thông qua hệ quản trị cơ sở dữ liệu MySQL với 10 bảng chính, thiết kế để theo dõi chặt chẽ mọi giao dịch chat ngang hàng:

1. users: Quản lý thông tin tài khoản định danh người dùng.
2. peers: Lưu trữ danh bạ phân tán gồm ID, Host, Port TCP, Port Web và trạng thái hoạt động cuối cùng (last_seen).
3. peer_status_logs: Nhật ký ghi nhận lịch sử thay đổi trạng thái online/offline của từng peer.
4. chat_groups: Lưu thông tin các nhóm chat được tạo ra trên mạng P2P, định danh chủ nhóm (owner_peer_id).
5. group_members: Bảng trung gian thể hiện quan hệ nhiều - nhiều giữa Peer và Group.
6. direct_messages: Lưu trữ lịch sử tin nhắn cá nhân trực tiếp (Direct) và tin nhắn Broadcast.
7. group_messages: Lưu trữ lịch sử tin nhắn của các nhóm chat.
8. offline_messages: Hàng đợi lưu trữ tin nhắn gửi tạm thời khi peer đích offline (phục vụ cơ chế Store-and-Forward).
9. message_acks: Theo dõi trạng thái ACK (đã nhận) cho từng tin nhắn, số lần thử lại (retries) và mã lỗi nếu gửi thất bại.
10. file_transfers: Lưu trữ metadata của các tệp tin được truyền nhận trực tiếp qua TCP.

4.4. Quy trình cài đặt chi tiết

Để triển khai hệ thống Chat P2P thành công trên môi trường cục bộ (Local), người quản trị hoặc sinh viên thực hiện theo các bước cụ thể sau:

Bước 1: Chuẩn bị môi trường hệ thống

Đảm bảo máy tính đã cài đặt Node.js phiên bản từ v18.0.0 trở lên và công cụ quản lý thư viện npm. (Kiểm tra bằng lệnh `node -v` và `npm -v` trên Terminal).

Khởi động hệ quản trị cơ sở dữ liệu MySQL Server và đảm bảo cổng kết nối mặc định 3306 đang hoạt động.

Bước 2: Tải dự án và cài đặt thư viện phụ thuộc

Mở terminal tại thư mục gốc của dự án (HTPT hoặc HeThongPhanTan) và chạy câu lệnh cài đặt các package cần thiết:

npm install

Lưu ý: Lệnh này sẽ tải toàn bộ các thư viện được định nghĩa trong package.json như `express`, `socket.io`, `mysql2`, `ejs`, `multer`, và `dotenv` vào thư mục `node_modules`.

Bước 3: Tạo tệp cấu hình môi trường `.env`

Sao chép cấu hình từ file mẫu `.env.example` để tạo tệp tin `.env` chạy chính thức:

Trên hệ điều hành Windows (PowerShell/CMD):

copy .env.example .env

Trên macOS hoặc Linux:

cp .env.example .env

Mở tệp `.env` vừa tạo và sửa đổi mật khẩu MySQL (`MYSQL_PASSWORD`), cũng như khóa bí mật `P2P_SHARED_SECRET` phù hợp với cấu hình của hệ thống máy tính.

Bước 4: Khởi tạo cơ sở dữ liệu MySQL

Sử dụng công cụ dòng lệnh của MySQL hoặc các phần mềm quản trị trực quan (như MySQL Workbench, phpMyAdmin, DBeaver) để tạo database và import cấu trúc bảng từ file schema có sẵn:

mysql -u root -p -e "CREATE DATABASE IF NOT EXISTS p2p_chat;"

mysql -u root -p p2p_chat < database/schema.sql

(Nếu cài đặt thành công, database `p2p_chat` sẽ hiển thị đầy đủ 10 bảng dữ liệu đã được định nghĩa).

4.5. Khởi chạy và vận hành hệ thống

Hệ thống cung cấp hai phương pháp khởi chạy linh hoạt để phục vụ cho cả quá trình bảo vệ đồ án trực quan (dùng Launcher UI) lẫn việc gỡ lỗi nâng cao bằng terminal.

4.5.1. Cách 1: Sử dụng Bootstrap Launcher UI

Phương pháp này tích hợp giao diện quản lý đồ họa trên trình duyệt giúp khởi tạo nhanh nhiều peer cùng lúc mà không cần mở nhiều cửa sổ terminal.

Bước 1: Khởi chạy Bootstrap Server và Launcher UI từ terminal:

npm run bootstrap

Bước 2: Mở trình duyệt web và truy cập địa chỉ: <http://127.0.0.1:3000>.

Bước 3: Tại giao diện Bootstrap Launcher, điền thông tin để tạo Peer mới tại biểu mẫu "Start New Peer":

- Peer ID: peer-a
- Username: Alice
- TCP Port: 5101
- Web Port: 3101
- Host IP: 127.0.0.1
- Nhấn nút: Start Peer.

Bước 4: Tiếp tục thực hiện tương tự để khởi tạo thêm hai peer tiếp theo:

- peer-b / Bob / TCP: 5102 / Web: 3102
- peer-c / Carol / TCP: 5103 / Web: 3103

Bước 5: Trên giao diện Launcher sẽ xuất hiện danh sách các peer đang hoạt động kèm theo nút Open UI. Nhấn vào nút Open UI tương ứng hoặc truy cập trực tiếp các địa chỉ dưới đây để mở Web UI điều khiển của từng người dùng:

- Alice UI: <http://127.0.0.1:3101>
- Bob UI: <http://127.0.0.1:3102>
- Carol UI: <http://127.0.0.1:3103>

4.5.2. Cách 2: Khởi chạy thủ công thông qua Terminal

Khi cần quan sát log luồng dữ liệu socket chi tiết từ cửa sổ điều khiển, bạn có thể khởi chạy riêng biệt từng tiến trình.

Cửa sổ Terminal 1 (Chạy Bootstrap Server):

npm run bootstrap

Cửa sổ Terminal 2 (Khởi động Peer A - Alice):

npm run peer:a

Cửa sổ Terminal 3 (Khởi động Peer B - Bob):

```
npm run peer:b
```

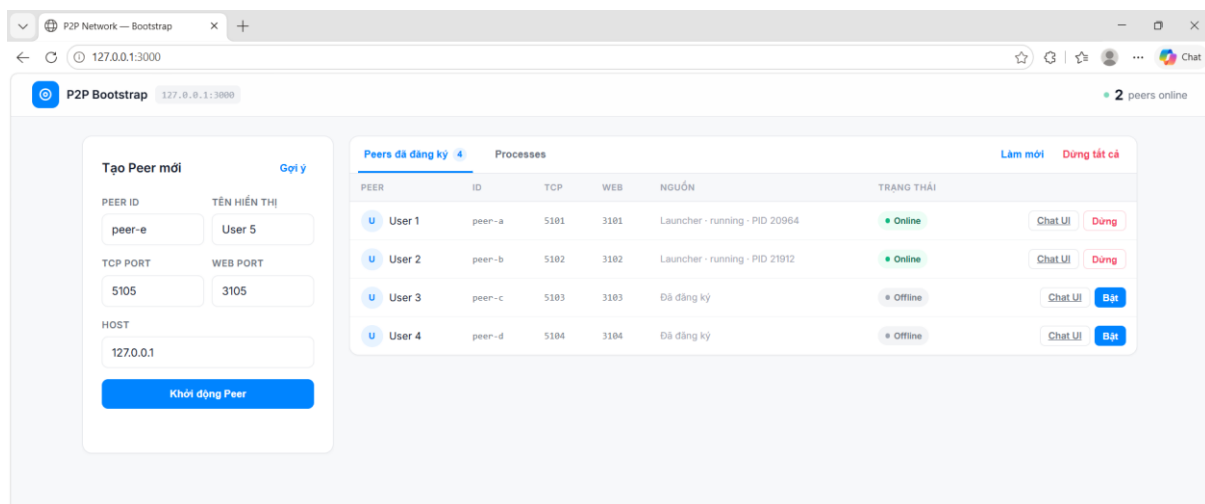
Cửa sổ Terminal 4 (Khởi động Peer C - Carol):

```
npm run peer:c
```

CHƯƠNG V : KẾT QUẢ VÀ DEMO

5.1 Giao diện Bootstrap Launcher

Bootstrap Launcher UI cho phép quản lý peer từ giao diện web. Người dùng có thể tạo peer mới bằng cách nhập Peer ID, Username, TCP Port, Web Port rồi bấm “Start Peer”.

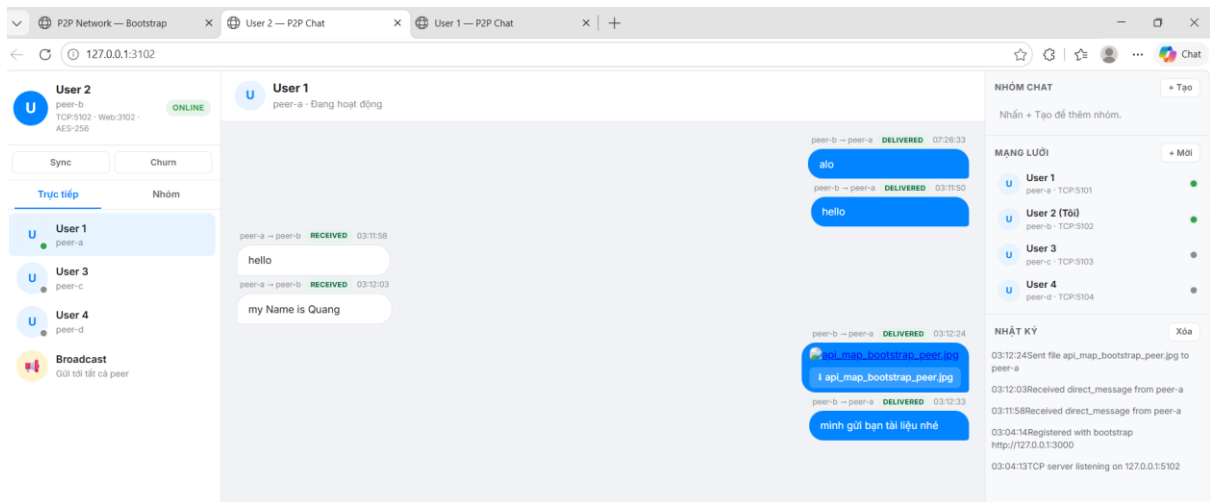


Hình 5.1. Giao diện trang bootstrap

Giao diện P2P Bootstrap hiển thị:

- Form tạo peer mới với các trường: Peer ID, Tên hiển thị, TCP Port, Web Port, Host.
- Danh sách peers đã đăng ký (4 peers) với thông tin: ID, ports, nguồn gốc, trạng thái (online/offline).
- Nút điều khiển cho từng peer: Chat UI, Dừng/Bật.
- Nút chức năng: Làm mới, Dừng tất cả.
- Trạng thái tổng quan: 2 peers online.

5.2 Giao diện Peer Web UI

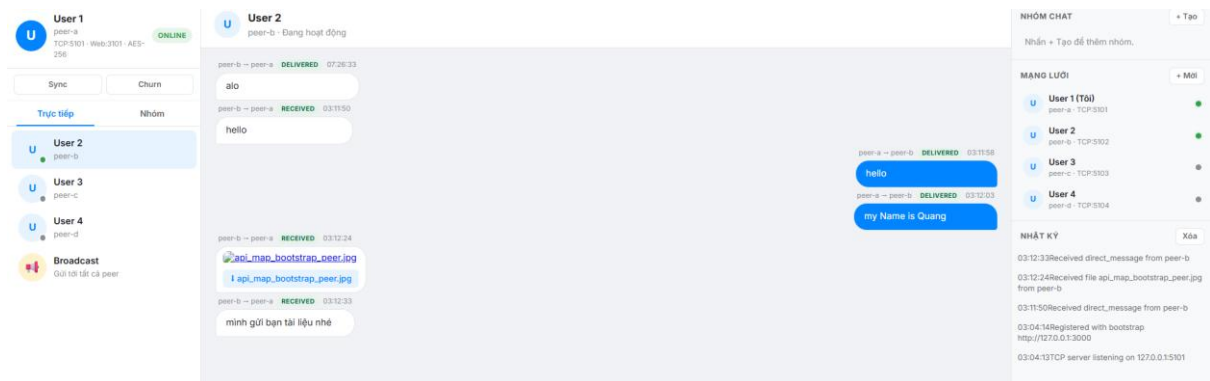


Hình 5.2. Giao diện trang Peer

Giao diện Chat P2P (User 2) hiển thị:

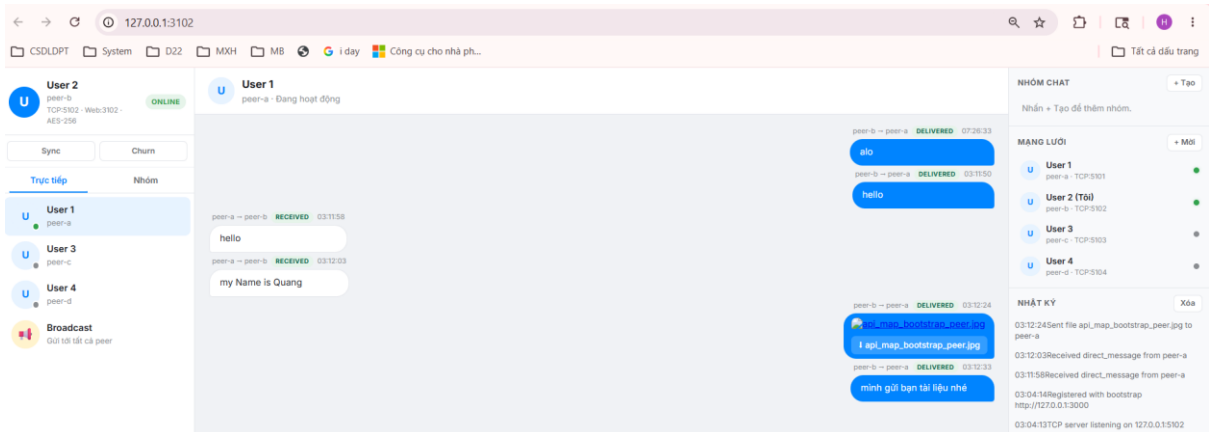
- Thông tin profile User 2 (Online, Port, mã hóa) và danh sách liên hệ (User 1, 3, 4, Broadcast).
- Khu vực chat trực tiếp với User 1: Tin nhắn văn bản, gửi file ảnh, trạng thái Deliver/Received.
- Panel bên phải: Quản lý nhóm chat, danh sách mạng lưới (Network) và nhật ký sự kiện (Log).

5.3. Demo gửi tin nhắn trực tiếp (Direct Message)



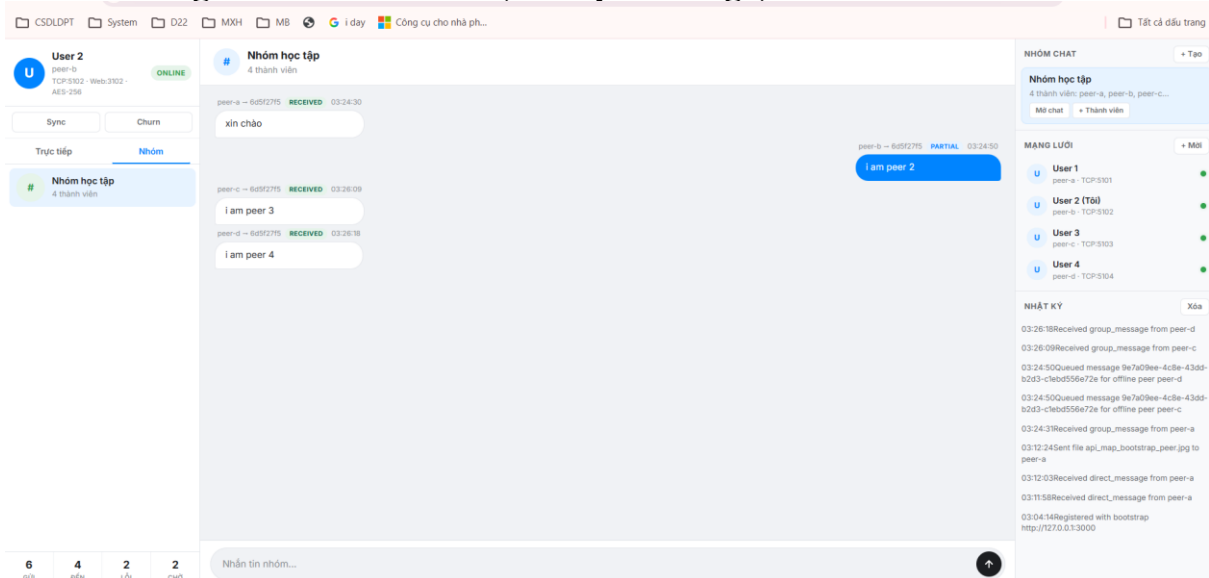
Hình 5.3: Peer A gửi tin nhắn trực tiếp cho Peer B

Hình 5.3 cho thấy Peer A gửi tin nhắn trực tiếp cho Peer B. Trạng thái hiển thị “delivered”, nghĩa là Peer B đã nhận được và trả ACK thành công.



Hình 5.4: Peer B nhận được tin nhắn trực tiếp từ Peer A

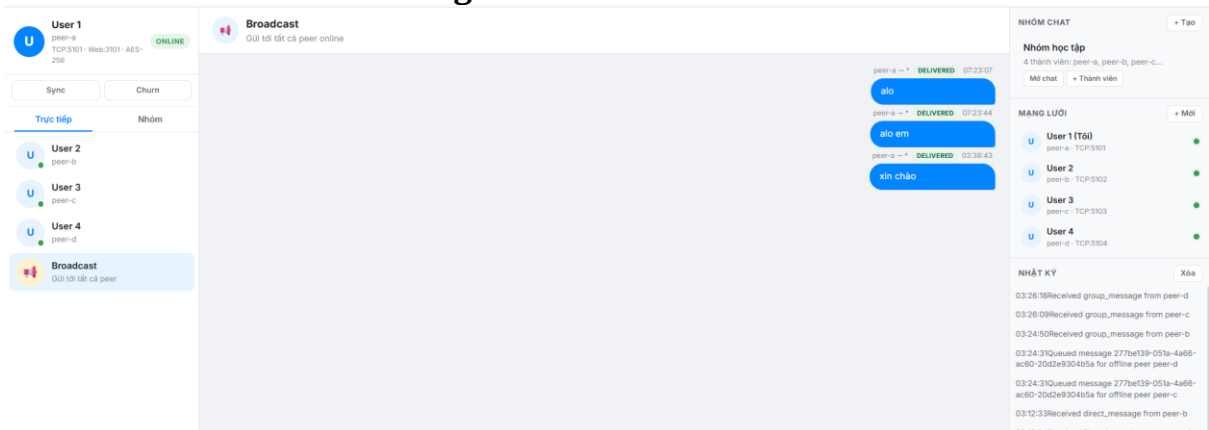
5.4. Demo gửi tin nhắn nhóm (Group Message)



Hình 5.5: Peer B nhận được tin nhắn trực tiếp từ Peer A

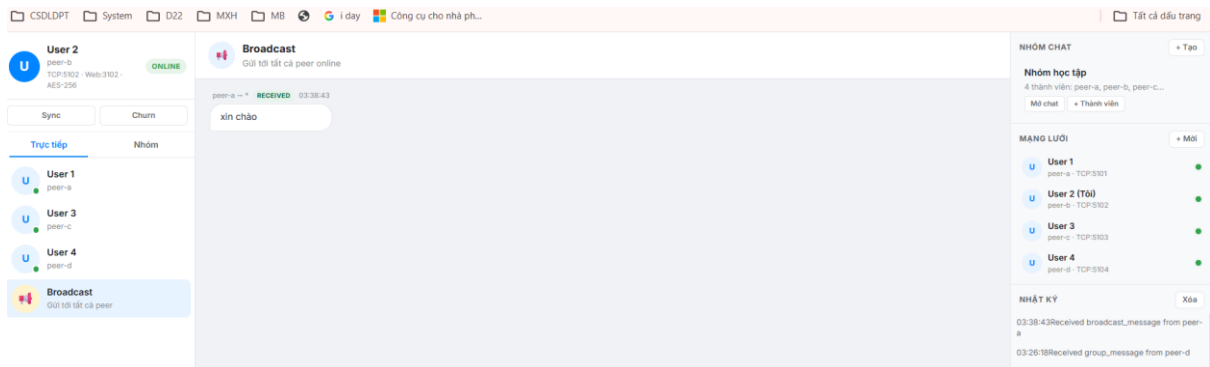
Hình 5.5 minh họa Peer A gửi tin nhắn nhóm. Tin nhắn được gửi trực tiếp qua TCP đến từng thành viên trong nhóm.

5.5 Demo Broadcast Message



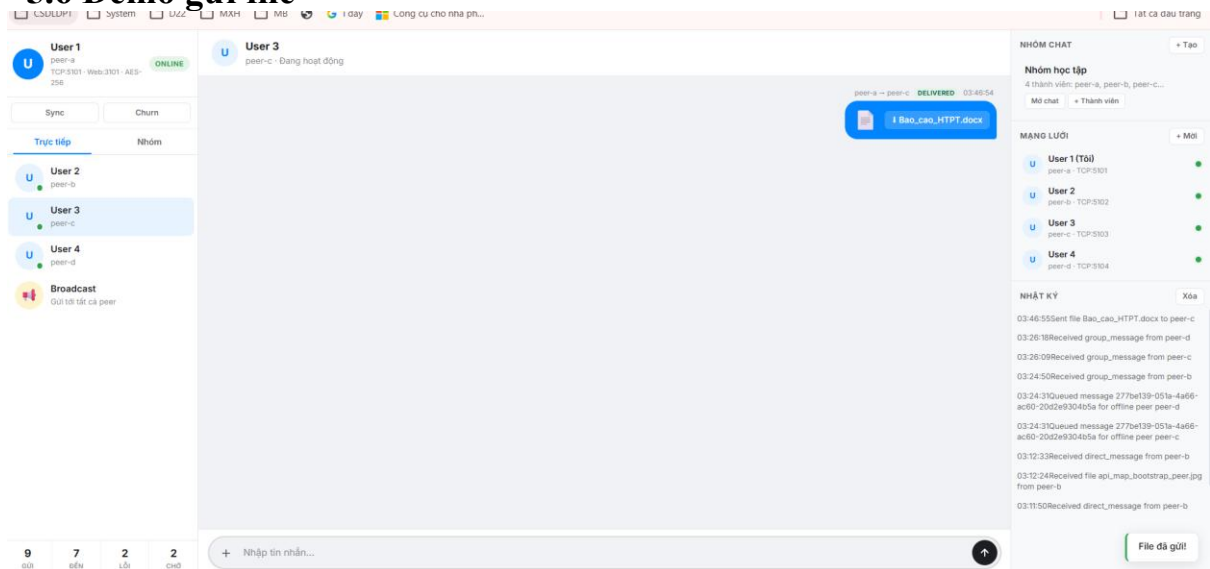
Hình 5.6: Peer A gửi tin nhắn Broadcast đến toàn bộ peer online

Hình 5.6 cho thấy Peer A sử dụng tab Broadcast để gửi tin nhắn đến tất cả peer đang online trong mạng.

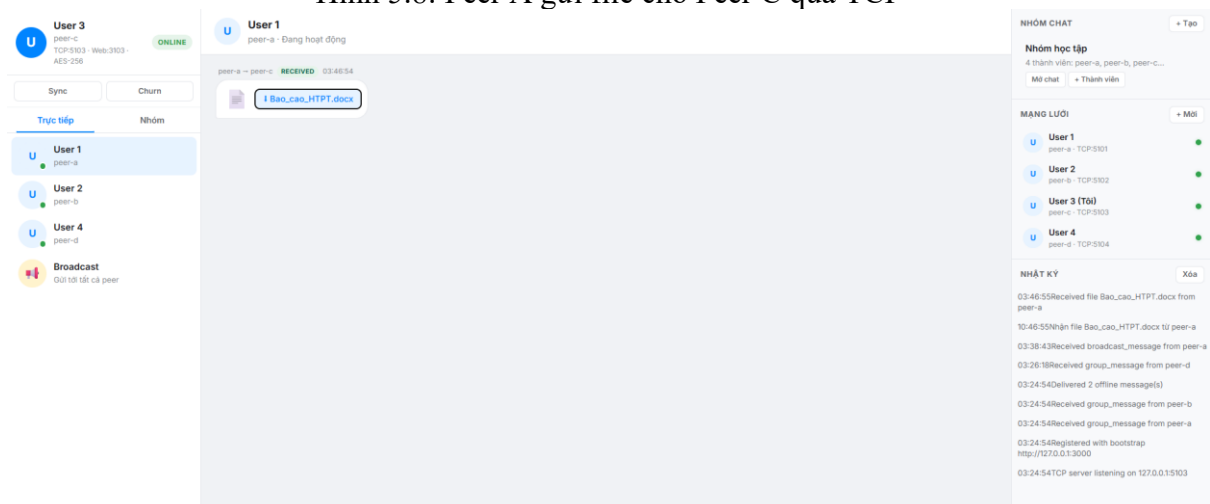


Hình 5.7: Peer C nhận được tin nhắn broadcast từ Peer A

5.6 Demo gửi file



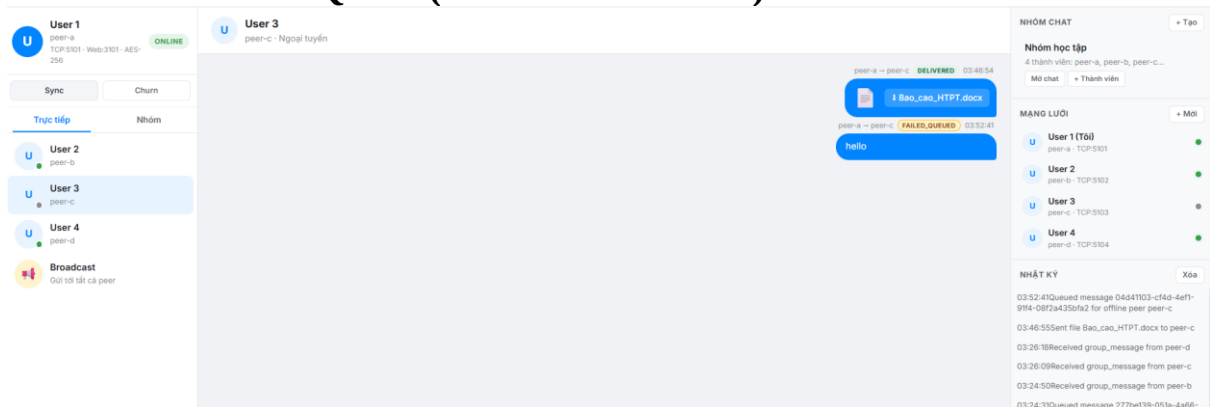
Hình 5.8: Peer A gửi file cho Peer C qua TCP



Hình 5.9: Peer C nhận được file từ Peer A

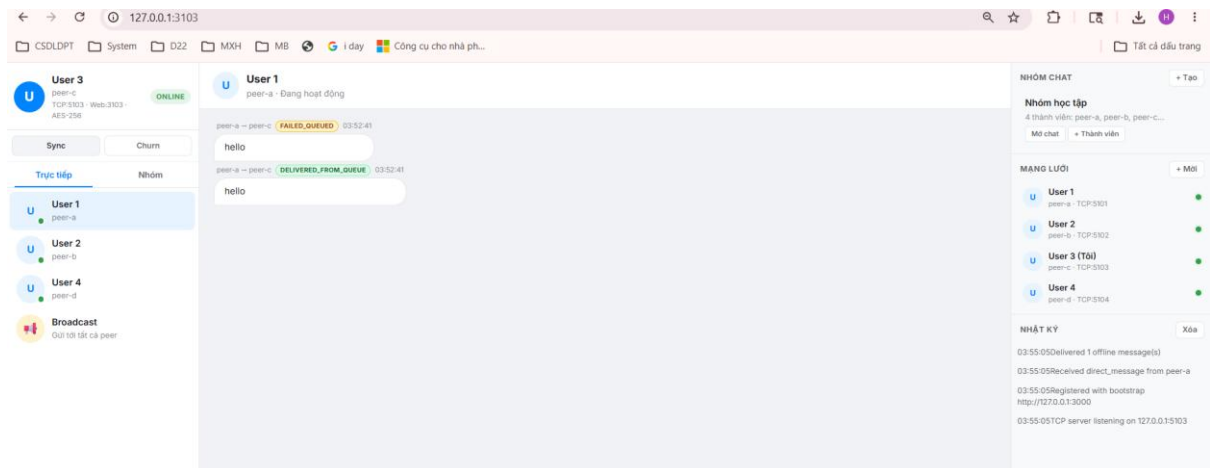
File được encode Base64, đóng gói vào payload TCP và gửi trực tiếp. Peer nhận decode và lưu vào thư mục received_files/. Metadata file transfer được ghi vào database

5.7. Demo Offline Queue (Store-and-Forward)



Hình 5.10: Peer A gửi tin cho Peer C đang offline – tin được queue

Hình 5.10 cho thấy khi Peer C offline, tin nhắn từ Peer A không gửi được qua TCP. Sau khi retry thất bại, hệ thống tự động queue tin nhắn trên bootstrap server. Trạng thái hiển thị “failed_queued”.



Hình 5.11: Peer C online lại và nhận tin nhắn từ offline queue

Hình 5.11 cho thấy khi Peer C online trở lại, nó tự động poll offline queue và nhận các tin nhắn đã được queue. System log hiển thị “Delivered X offline message(s)”.

5.8. Tổng hợp kết quả kiểm thử

STT	Chức năng	Kết quả	Ghi chú
1	Peer đăng ký với Bootstrap Server	✓ Đạt	Đăng ký thành công khi khởi động
2	Peer Discovery (tìm kiếm peer khác)	✓ Đạt	Đồng bộ danh sách peer mỗi 5 giây

3	Heartbeat duy trì trạng thái online	✓ Đạt	Gửi heartbeat mỗi 8 giây
4	Gửi tin nhắn trực tiếp (Direct Message)	✓ Đạt	ACK phản hồi theo thời gian thực
5	Gửi tin nhắn nhóm (Group Message)	✓ Đạt	Tin nhắn được gửi tới từng thành viên
6	Gửi tin nhắn Broadcast	✓ Đạt	Phân phối tới toàn bộ peer đang online
7	Gửi file qua giao thức TCP	✓ Đạt	Hỗ trợ file tối đa 10MB
8	Mã hóa AES-256-GCM	✓ Đạt	Mã hóa và giải mã hoạt động chính xác
9	Retry khi gửi thất bại	✓ Đạt	Tối đa 3 lần thử gửi lại
10	Offline Queue (Store-and-Forward)	✓ Đạt	Peer offline sẽ được poll mỗi 5 giây
11	Relay qua peer trung gian	✓ Đạt	Ưu tiên relay trước khi đưa vào queue
12	Churn Simulation	✓ Đạt	Mô phỏng peer online/offline liên tục
13	Web UI Realtime (Socket.IO)	✓ Đạt	Giao diện cập nhật tức thì
14	Launcher UI (Start/Stop Peer)	✓ Đạt	Sử dụng child process để quản lý peer
15	Lưu trữ dữ liệu bằng MySQL	✓ Đạt	Hệ thống gồm 11 bảng dữ liệu
16	Memory Fallback	✓ Đạt	Sử dụng bộ nhớ tạm khi MySQL không khả dụng

CHƯƠNG 6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết Quả Đạt Được

Đồ án đã hoàn thành xây dựng hệ thống chat ngang hàng P2P với đầy đủ các chức năng theo yêu cầu đề ra. Cụ thể:

- Kiến trúc P2P đúng nghĩa: Tin nhắn được truyền trực tiếp giữa các peer thông qua TCP socket. Bootstrap server chỉ đảm nhiệm vai trò hỗ trợ peer discovery và quản lý metadata, hoàn toàn không nằm trên đường truyền tin nhắn chính.
- Đầy đủ chức năng chat: Hệ thống hỗ trợ các hình thức giao tiếp direct message, group message, broadcast message và file transfer — tất cả đều vận hành trực tiếp qua kết nối TCP P2P.
- Xử lý peer offline: Cơ chế store-and-forward với hàng đợi offline trên bootstrap cho phép lưu trữ tin nhắn tạm thời và tự động giao lại khi peer kết nối trở lại.
- Bảo mật: Nội dung tin nhắn được mã hóa bằng chuẩn AES-256-GCM trước khi truyền qua mạng, đảm bảo tính bảo mật và toàn vẹn dữ liệu.
- Độ tin cậy: Cơ chế ACK, retry, timeout và relay qua peer trung gian được tích hợp nhằm đảm bảo tin nhắn đến đích ngay cả trong điều kiện mạng không ổn định.
- Giao diện thời gian thực: Web UI tích hợp Socket.IO, cập nhật tin nhắn, trạng thái kết nối và nhật ký hệ thống tức thì, mang lại trải nghiệm sử dụng mượt mà.
- Mô phỏng churn: Hệ thống hỗ trợ cơ chế peer tự động rời và tham gia mạng, phục vụ việc kiểm chứng tính ổn định trong các kịch bản thực tế.
- Launcher UI: Giao diện quản lý trực quan cho phép tạo và dừng peer một cách thuận tiện, phù hợp cho mục đích demo và thử nghiệm

6.2. Hạn Chế

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế cần ghi nhận:

- Mã hóa đối xứng đơn giản: Hệ thống hiện sử dụng một shared secret chung cho tất cả các peer, chưa triển khai cơ chế trao đổi khóa bất đối xứng như RSA hay Diffie-Hellman để đảm bảo mỗi cặp peer có khóa riêng độc lập.
- Giới hạn trong môi trường localhost: Hệ thống được thiết kế và tối ưu cho môi trường demo trên một máy đơn. Việc triển khai trên mạng thực tế đòi hỏi xử lý thêm các vấn đề NAT traversal và firewall.
- Hạn chế trong truyền file: Dung lượng file tối đa là 2MB; phương thức mã hóa Base64 làm tăng kích thước dữ liệu thêm khoảng 33%, chưa hỗ trợ streaming hay chunk transfer cho file lớn.
- Thiếu cơ chế xác thực: Hệ thống chưa có giải pháp xác thực danh tính peer. Bất kỳ client nào biết địa chỉ bootstrap server đều có thể đăng ký và tham gia mạng.
- Phụ thuộc vào bootstrap server: Bootstrap server vẫn là điểm tập trung duy

nhất. Nếu server này ngừng hoạt động, peer mới không thể gia nhập mạng và cơ chế offline queue cũng sẽ ngừng hoạt động theo.

6.3. Hướng Phát Triển

Trên cơ sở nền tảng đã xây dựng, một số hướng phát triển tiếp theo được đề xuất như sau:

- Mã hóa bất đối xứng: Triển khai RSA hoặc giao thức Diffie-Hellman để thiết lập khóa riêng cho từng cặp peer, nâng cao đáng kể mức độ bảo mật của hệ thống.
- NAT Traversal: Ứng dụng các kỹ thuật STUN/TURN hoặc UPnP nhằm cho phép hệ thống hoạt động ổn định trên môi trường Internet thực, vượt qua rào cản NAT và tường lửa.
- Distributed Hash Table (DHT): Thay thế bootstrap server tập trung bằng kiến trúc DHT theo thuật toán Kademlia, loại bỏ điểm lỗi đơn và tăng tính phi tập trung của hệ thống.
- Cải tiến truyền file: Bổ sung hỗ trợ chunk transfer, tính năng resume download và khả năng xử lý file dung lượng lớn hơn, hướng đến trải nghiệm người dùng thực tế hơn.
- Xác thực và phân quyền: Xây dựng hệ thống đăng nhập, cấp phát JWT token và phân quyền truy cập nhóm để kiểm soát thành viên trong mạng P2P.
- End-to-End Encryption nâng cao: Áp dụng Signal Protocol hoặc cơ chế Double Ratchet nhằm đảm bảo forward secrecy — bảo vệ các phiên giao tiếp cũ ngay cả khi khóa hiện tại bị lộ.
- Ứng dụng di động: Phát triển ứng dụng mobile trên nền tảng React Native hoặc Flutter, kết nối trực tiếp với peer backend để mở rộng khả năng tiếp cận.
- Tích hợp WebRTC: Kết hợp WebRTC vào kiến trúc hiện có để hỗ trợ truyền thông audio và video theo mô hình P2P, mở rộng hệ thống thành nền tảng giao tiếp đa phương tiện.

Tài liệu tham khảo

1. A. S. Tanenbaum and M. Van Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. Pearson, 2017.
2. R. Schollmeier, “A definition of peer-to-peer networking for the classification of peer-to-peer architectures and applications,” in *Proceedings of the First International Conference on Peer-to-Peer Computing*, IEEE, 2001.
3. National Institute of Standards and Technology (NIST), *Advanced Encryption Standard (AES)*, FIPS PUB 197, 2001.
4. M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST Special Publication 800-38D, 2007.
5. Node.js Documentation, “Net module.” [Online]. Available: <https://nodejs.org/api/net.html>. [Accessed: May 2026].
6. Express.js Documentation, “Express - Node.js web application framework.” [Online]. Available: <https://expressjs.com/>. [Accessed: May 2026].
7. Socket.IO Documentation, “Socket.IO - Bidirectional and low-latency communication.” [Online]. Available: <https://socket.io/docs/v4/>. [Accessed: May 2026].
8. MySQL Documentation, “MySQL 8.0 Reference Manual.” [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>. [Accessed: May 2026].
9. J. Postel, *Transmission Control Protocol*, RFC 793, Sep. 1981.
10. I. Fette and A. Melnikov, *The WebSocket Protocol*, RFC 6455, Dec. 2011.
11. J. F. Buford, H. Yu, and E. K. Lua, *P2P Networking and Applications*. Morgan Kaufmann, 2009.
12. E. K. Lua, J. Crowcroft, M. Pias, R. Sharma, and S. Lim, “A survey and comparison of peer-to-peer overlay network schemes,” *IEEE Communications Surveys & Tutorials*, vol. 7, no. 2, pp. 72–93, 2005.